Optimizing Multi-GPU Hash Tables under Unified Virtual Memory Oversubscription

CS636 — Analysis of Concurrent Programs
Course Project Report

https://github.com/HavisH-R/CS636-Multi-GPU-Hashtable

Harsh Pratap Singh Rayavarapu Viswa Sai Havish

Contents

1 Introduction **5**
1.1 Unified Virtual Memory and the Oversubscription Promise . . . 5
1.2 Why Hash Tables Are the Worst Case for UVM . . . 5
1.2.1 The Spatial Locality Mismatch . . . 5
1.2.2 The Amplification Factor . . . 6
1.3 The Central Mismatch . . . 6
1.4 Project Contributions . . . 6
1.5 Relationship to Prior Work . . . 7

2 Hardware and Software Environment **7**
2.1 Compute Nodes . . . 7
2.2 Oversubscription Enforcement: 90% VRAM Reserved . . . 7
2.3 GPU-GPU Communication . . . 9

3 The Oversubscription Problem: Measurements and Analysis **9**
3.1 Workload Configuration . . . 9
3.2 The In-Memory Ceiling . . . 9
3.3 The Oversubscription Cliff . . . 10
3.4 The Four Interlocking Fault Mechanisms . . . 10
3.4.1 Mechanism 1: Synchronous Page Fault . . . 10
3.4.2 Mechanism 2: Warp Stall — All-or-Nothing Fault Decision . . . 10
3.4.3 Mechanism 3: SM Drainage . . . 11
3.4.4 Mechanism 4: Fault-Migrate-Replay . . . 11
3.5 Naive Multi-GPU: The Ping-Pong Catastrophe . . . 11
3.6 Hinted Multi-GPU: The Latency Trap . . . 12

4 The Hash Table Kernel: Design and Implementation **13**
4.1 Data Structure: Open-Addressed Flat Array . . . 13
4.2 MurmurHash: Why Not Simple Modulo . . . 14
4.3 Warp-Cooperative Insert Protocol . . . 14

4.3.1 Why Assign 32 Threads to One Insert . . . 16
4.4 Kernel Launch Configuration . . . 16
4.5 CUDA Memory Allocation Strategy . . . 16

5 CPU-Level Schedulers: Exploring the Scheduling Axis 17
5.1 Configuration . . . 17
5.2 Work-Stealing: 63.958 M Ops/s . . . 18
5.2.1 Hypothesis . . . 18
5.2.2 Why It Works: Key Buffer Shrinkage . . . 18
5.2.3 The Naming Irony . . . 18
5.2.4 Nsight Memory Traffic . . . 19
5.3 Deep Pipeline: 80.532 M Ops/s . . . 19
5.3.1 Motivation . . . 19
5.3.2 The CPU/GPU Ratio Insight . . . 20
5.3.3 Measured Results . . . 21
5.3.4 Why It Still Cannot Reach the Ceiling . . . 21
5.4 Multistream: 29.647 M Ops/s . . . 21
5.4.1 In-Memory Rationale . . . 22
5.4.2 Two Failures Under Oversubscription . . . 22
5.5 Pipeline: 23.855 M Ops/s . . . 23
5.6 Adaptive Batch Sizing: 23.841 M Ops/s . . . 24
5.6.1 Design . . . 24
5.6.2 Failure: Initialization Bug . . . 24
5.6.3 Correct Behavior (1M Initial Batch) . . . 25
5.7 Intelligent Prefetch: 2.123 M Ops/s . . . 25
5.7.1 Research Foundation . . . 25
5.7.2 Implementation . . . 25
5.7.3 The Hidden UVM Rule . . . 26
5.7.4 Nsight Evidence . . . 26

6 Radix Routing: The Three-Phase Solution 27
6.1 Architectural Inspiration: The Macro-Cache Analogy . . . 27
6.2 Core Insight: Partition Alignment . . . 27
6.3 Phase 1: GPU Routing (Histogram + Scatter) . . . 28
6.4 Phase 2: Local Radix Partitioning . . . 29
6.4.1 Why B=128 Beats B=64 . . . 30
6.5 Phase 3: Bucketed Insertion . . . 30
6.5.1 Numeric Walkthrough . . . 30
6.6 UVM Advice and Prefetch Setup . . . 31
6.7 Full Measured Results: Radix 64 Buckets . . . 31
6.8 Overhead Breakdown . . . 32
6.9 Super-Radix: Pipelining the Routing Overhead . . . 33
6.10 WCME-Pipeline Branch . . . 33

7 Comparative Analysis 34
7.1 Complete Performance Hierarchy . . . 34
7.2 Root Cause Classification . . . 34

7.3 The Performance Hierarchy Inversion . . . 35

8 Additional Experiments: What Else We Tried 35
8.1 Hot-Cold Tracking (scheduler-hotcold) . . . 35
8.2 GPU-GPU Load Balancing (scheduler-gpu-loadbalance) . . . 37
8.3 Remote-Operation Queuing (scheduler-queues) . . . 37
8.4 Score-Embedded Eviction (scheduler-hkv) . . . 37
8.5 Warp-Cooperative Match-and-Elect (WCME) . . . 38
8.6 Future Work . . . 38

9 Related Work 39

10 Conclusion 39

A Build and Run Instructions 40
A.1 Prerequisites . . . 40
A.2 Repository Setup . . . 40
A.3 Common Runtime Flags . . . 41
A.4 Branch-by-Branch Build & Run Reference . . . 41
A.4.1 Baselines . . . 41
A.4.2 CPU-Level Schedulers (single GPU, oversubscription 9.1×) . . . 42
A.4.3 Multi-GPU Radix Routing (the solution) . . . 43
A.4.4 Advanced / Experimental Branches . . . 44
A.5 Profiling with Nsight Systems . . . 44
A.6 Reproducing All Results (Quick Reference) . . . 45

B CUDA API Reference 46
B.1 Memory Allocation . . . 46
B.2 UVM Memory Advice . . . 46
B.3 Async Prefetch . . . 46
B.4 Synchronization . . . 47
B.5 Warp Intrinsics . . . 47

C Branch Summary 47

Abstract

We investigate the performance of open-addressed GPU hash tables under Unified Virtual Memory (UVM) oversubscription on dual NVIDIA GeForce GTX 1080 hardware connected via PCIe 3.0 (no NVLink). To simulate oversubscription, a 7,293 MB dummy allocation consumes **89% of each GPU's 8,192 MB VRAM**, leaving only ∼899 MB free—creating a controlled 9.1× oversubscription ratio for all experiments. The canonical single-GPU oversubscribed configuration achieves only 2.77 M Ops/s against a fully in-memory ceiling of 354.7 M Ops/s—a 128× collapse driven by synchronous page faults, warp stalling, SM drainage, and the fault-migrate-replay cycle. Naive multi-GPU execution further degrades to 17.011 M Ops/s owing to 38× PCIe traffic amplification from page thrashing between GPUs.

We implement and benchmark seven CPU-level schedulers (multistream, pipeline, adaptive, work-stealing, deep pipeline, intelligent prefetch) and a three-phase multi-GPU radix routing strategy across ten Git branches. The radix routing solution achieves **174.672 M Ops/s** (64 buckets) and **223.487 M Ops/s** (128 buckets), recovering 49–63% of the in-memory ceiling and outperforming the naive multi-GPU baseline by 7.1–13.1×. An extended Super-Radix variant with pipelined routing reduces the pre-processing overhead from 52% to 17% of wall-clock time.

The central finding is that UVM oversubscription in hash-table workloads is a *capacity* problem, not a throughput or scheduling problem. The only effective interventions are those that reduce per-GPU working set: data partitioning (Radix), key-buffer shrinkage (Work-Stealing), or their combination. All pure scheduling techniques (pipelining, multistream, prefetch) address the wrong problem and produce marginal or negative results.

All measurements are from NVIDIA Nsight Systems profiles on physical hardware running Linux.

1 Introduction

This report accompanies the CS636 — Analysis of Concurrent Programs course project. It follows a deliberate build-up narrative: Act I establishes the problem and hardware foundation; Act II shows CPU-level scheduling attempts on a single GPU and why they address the wrong axis; Act III reveals the root-cause solution (radix routing, multi-GPU); Act IV provides comparative analysis. The best results intentionally appear last.

1.1 Unified Virtual Memory and the Oversubscription Promise

Unified Virtual Memory (UVM), introduced in CUDA 6 and substantially extended through Pascal (2016) and Volta (2017), provides a single virtual address space shared across the CPU and all attached GPUs. From the programmer's perspective, a pointer allocated with `cudaMallocManaged` can be dereferenced from any processor without explicit `cudaMemcpy` calls. The CUDA driver and GPU hardware MMU transparently manage data residency.

UVM's primary value proposition is **oversubscription**: the ability to run programs whose combined working set exceeds the physical VRAM of any single GPU. When a GPU thread accesses a virtual address whose backing physical page is not resident in VRAM, the GPU hardware fires a *page fault interrupt*. The CPU-side UVM driver services the fault by:

1. Identifying the faulting page's current physical location (host RAM or a remote GPU)
2. Finding a free VRAM page (evicting an existing page if necessary)
3. Issuing a DMA transfer over PCIe to migrate the 4 KB page
4. Updating the GPU page tables
5. Allowing the faulting instruction to *replay*

This enables programs that are impossible on a single device but comes with a steep performance penalty when faults occur at high frequency.

1.2 Why Hash Tables Are the Worst Case for UVM

Hash tables are the adversarial access pattern for every caching layer in the memory hierarchy. A well-designed hash function (such as a MurmurHash-style integer finalizer) scatters keys *uniformly* across all table slots—exactly the property that minimizes collisions. But uniformity is precisely what memory systems hate: it maximizes the number of distinct pages touched per unit of work.

1.2.1 The Spatial Locality Mismatch

Every memory system from L1 cache to UVM bets on *spatial locality*: if you access address X, you will probably access $X + 1$, $X + 2$, $X + 3$ soon. It amortizes transfer cost by moving data in large granules:

For a hash table with 8-byte slots, each UVM page holds 512 slots. Sequential array access serves 512 consecutive accesses from a single page fault (100% transfer efficiency). Hash table access scatters probes across all ∼174,000 pages of a 678 MB table—each access lands on a new page, achieving <0.2% transfer efficiency in the worst case.

Memory Level	Transfer Unit	Miss Penalty	Winning Pattern
L1 cache	128 B cache line	∼28 cycles	Sequential arrays
L2 cache	128 B cache line	∼200 cycles	Loop tiling
DRAM	4–8 KB row	∼600 cycles	Blocked access
UVM / PCIe	4 KB page	**∼50,000–80,000 cycles**	*(impossible for hash tables)*

Table 1: Memory hierarchy transfer granularities and miss penalties.

1.2.2 The Amplification Factor

$$\text{Amplification} = \frac{\text{Page size}}{\text{Useful bytes per access}} = \frac{4096\text{ B}}{8\text{ B}} = 512\times \tag{1}$$

Under UVM oversubscription with random access, the system must move 512 bytes for every 1 byte of hash table data actually needed. For 80M inserts across a 678 MB table, this translates to *measured* 24.4 GB of PCIe traffic in the naive multi-GPU case—38× amplification of the 640 MB actual workload.

1.3 The Central Mismatch

GPU Streaming Multiprocessors (SMs) hide memory latency by maintaining thousands of warps in-flight: when one warp stalls waiting for DRAM (∼600 cycles, ∼375 ns at 1.6 GHz), the scheduler switches to another ready warp. This *occupancy-based latency hiding* keeps the SM compute pipeline busy as long as the warp pool is large enough to cover the stall window.

UVM page faults break this mechanism. A fault costs ∼20–50 μs (including PCIe DMA, driver interrupt handling, page table update, and TLB shootdown), or roughly ∼32,000–80,000 GPU cycles at the GTX 1080's 1.6 GHz boost clock—orders of magnitude worse than DRAM. An SM with 2,048 active warps at 0.5% fault probability generates faults at a rate that exceeds the UVM driver's servicing capacity. The ready-warp queue drains, compute cores go idle, and throughput collapses to near zero:

$$\text{Stall window per fault} \approx 50{,}000\text{–}80{,}000\text{ cycles} \gg \text{SM warp pool} \approx 64\text{ warps} \times 20\text{ cycles} = 1{,}280\text{ cycles} \tag{2}$$

1.4 Project Contributions

This project makes the following contributions:

1. **Empirical characterization** of the UVM oversubscription cliff on hash tables: 354.7 M Ops/s in-memory → 2.77 M Ops/s at 9.1× oversubscription (128× collapse).

2. **Seven CPU-level scheduler implementations** across independent Git branches, demonstrating that scheduling alone cannot overcome the 419 MB VRAM deficit.

3. **Discovery of the CPU-reads-UVM anti-pattern**: CPU accesses to managed memory migrate pages from GPU VRAM to host RAM, destroying the page cache.

4. **Three-phase radix routing pipeline** that eliminates cross-GPU page migration by pre-routing keys to their owning GPU before any insert kernel executes, achieving 174.672 M Ops/s (7.1×) and 223.487 M Ops/s (13.1×) over the naive baseline.

5. **Root cause analysis** demonstrating that oversubscription is a *capacity* problem: only strategies that reduce per-GPU working set are effective.

1.5 Relationship to Prior Work

WarpDrive (IEEE 2018) [1] is the closest prior work in multi-GPU hash table design. It partitions the hash table across GPUs and assigns ownership—the same logical structure as this project. However, WarpDrive operates in a fundamentally different regime:

Dimension	WarpDrive	This Project
Interconnect	NVLink (100–300 GB/s, ∼1 μs)	PCIe 3.0 (∼16 GB/s, 1–2 μs)
Memory model	Explicit `cudaMemcpy`	UVM (driver-managed)
Data fits in VRAM?	Yes	No (oversubscription is the focus)
Data movement	Bulk pre-planned (Gossip library)	On-demand page faults (unless prevented)
Problem addressed	Throughput scaling	Correctness + performance when data overflows

Table 2: Comparison with WarpDrive (IEEE 2018).

DACHash (2021) [2] inspired our radix pre-sorting approach: it used radix reordering to avoid 200-cycle L2 cache misses. We repurposed the same insight at the UVM level—a UVM page is a 50,000-cycle "macro cache line," and the same reordering technique that avoids L2 misses also avoids UVM faults.

Hive Hash Table (2025) [3] introduced the `__match_any_sync` WCME pattern we implemented in the `scheduler-wcme-pipeline` branch.

HierarchicalKV / Merlin (NVIDIA 2024) [4] provided the score-embedded eviction concept used in the `scheduler-hkv` branch.

SAFE (IEEE CAL 2025) [5] and **MSched** (arXiv 2025) [6] were the research basis for the intelligent prefetching scheduler, which ultimately failed due to the CPU-reads-UVM anti-pattern.

2 Hardware and Software Environment

2.1 Compute Nodes

All experiments were conducted on **gpu0**, the GPU compute server at IIT Kanpur (accessed as `rviswas22@gpu0`).

2.2 Oversubscription Enforcement: 90% VRAM Reserved

All oversubscribed experiments allocate a **7,293 MB dummy array** with `cudaMallocManaged`, consuming approximately **89% of each GPU's 8,192 MB VRAM**. This leaves only ∼899 MB free per GPU, creating a controlled **9.1× oversubscription ratio** (total VRAM / free VRAM):

Component	Specification
Server	**gpu0** (IIT Kanpur GPU cluster)
Login	`rviswas22@gpu0`
GPU model	NVIDIA GeForce GTX 1080 (×2)
Architecture	Pascal (sm_61)
VRAM per GPU	8,192 MB GDDR5X
Oversubscription dummy	7,293 MB (= **90% of VRAM reserved**)
Free VRAM (experiments)	∼899 MB per GPU
L2 cache per GPU	2,048 KB
CUDA cores per GPU	2,560 (20 SMs × 128 cores)
Max warps per SM	64 (2,048 threads)
Max concurrent kernels	32 (sm_61 hardware limit; `concurrentKernels`=1 is a boolean capability flag)
GPU interconnect	PCIe 3.0 ×16 (no NVLink)
Peak GPU bandwidth	∼320 GB/s (intra-GPU DRAM)
PCIe bandwidth	∼16 GB/s peak (host↔device)
P2P enabled	GPU 0 ↔ GPU 1 (both directions, verified)
CUDA version	12.x
Compiler flags	`-std=c++17 -arch=sm_61 -O3`
Profiler	NVIDIA Nsight Systems (`nsys`)

Table 3: Hardware and software environment.

$$\text{Total VRAM per GPU} = 8{,}192 \text{ MB} \tag{3}$$

$$\text{Dummy allocation} = 7{,}293 \text{ MB} \quad (= 89.0\% \text{ of VRAM}) \tag{4}$$

$$\text{Free VRAM} = 8{,}192 - 7{,}293 = \mathbf{899} \text{ MB} \tag{5}$$

$$\text{Oversubscription ratio} = \frac{8{,}192 \text{ MB total}}{899 \text{ MB free}} \approx \mathbf{9.1}\times \tag{6}$$

The dummy array is touched with `cudaMemset` to force page residency before any experiment begins:

Listing 1: Oversubscription: 89% VRAM deliberately reserved.

```
// Reserve 89% of VRAM (7293 MB of 8192 MB per GPU)
size_t dummy_size = (size_t)7293 * 1024 * 1024;
cudaMallocManaged(&dummy_array, dummy_size,
                  cudaMemAttachGlobal);
// Touch to force page residency -- ensure pages are
// actually in VRAM before the experiment starts
cudaMemset(dummy_array, 0, dummy_size);
cudaDeviceSynchronize();

// Verify: only ~899 MB remains free
// cudaMemGetInfo(&free, &total)
// free ~= 899 MB  (11% of 8192 MB)
```

This design choice ensures that the oversubscription condition is *deterministic and reproducible* across all runs. Every scheduler is evaluated under identical VRAM pressure.

2.3 GPU-GPU Communication

Two GTX 1080s connected via PCIe 3.0 communicate exclusively through the PCIe switch (CPU root complex). There is no direct GPU-to-GPU wire. `cudaMemcpy(DeviceToDevice)` between the two GPUs traverses the path: GPU 0 VRAM → PCIe → CPU root complex → PCIe → GPU 1 VRAM, with ∼16 GB/s effective bandwidth. P2P direct access is enabled via `cudaDeviceEnablePeerAccess(1, 0)` and its reciprocal, allowing UVM pages to migrate between GPUs without CPU involvement.

3 The Oversubscription Problem: Measurements and Analysis

Presentation: Act I — The Problem (slides s2, s3, s3b, sBaseline)

3.1 Workload Configuration

Parameter	Value
Operations	80,000,000 inserts (no lookups or deletes)
Key type	`uint32_t` (4 bytes)
Value type	`uint32_t` (4 bytes)
Pair size	8 bytes
Input buffer	80M × 8 B = 640 MB
Hash table capacity	89,008,207 slots
Table size	89M × 8 B ≈ 678 MB
Load factor	80M/89M ≈ 89.9%
Hash function	Integer finalizer mix (WarpCore variant, borrowed from WarpCore)
Collision resolution	Open addressing, warp-stride-32 probing
Key distribution	Uniform random

Table 4: Workload parameters, fixed across all experiments.

Why both allocations must be live simultaneously. The input buffer (640 MB) holds pending `(key, value)` pairs—the work queue. The hash table (678 MB) holds the destination slots—the data structure. During kernel execution, every thread reads `input[i]`, computes `murmur_mix(key) % capacity`, and writes to `table[slot]`. Both must be addressable throughout the kernel's lifetime. The input buffer cannot be freed mid-kernel because threads at different progress points are still consuming it.

$$\text{Total working set} = 640\ \text{MB (input)} + 678\ \text{MB (table)} = 1{,}318\ \text{MB} \tag{7}$$

$$\text{VRAM deficit} = 1{,}318\ \text{MB} - 899\ \text{MB} = \mathbf{419}\ \text{MB} \quad (32\%\ \text{always paged}) \tag{8}$$

3.2 The In-Memory Ceiling

Without the oversubscription dummy allocation, all data fits in 8,192 MB VRAM. The Nsight Systems profile shows pure compute-bound execution:

Listing 2: Nsight Systems output – single GPU in-memory (354.7 M Ops/s).

```
Kernel: _Z23batch_insert_gpu_kernelP11key_value_sS0_mmmjm
  Single run:     225.723 ms  ->  354.7 M Ops/s
  Grid:           5,000,000 x 1 x 1  @  512 threads/block
  Threads total:  2,560,000,000  (= 80M ops x 32 threads/warp)

HtoD: 1,280.000 MB / 612 transfers (avg 2.092 MB)  -- prefetch
DtoH:     0 MB  <-- CRITICAL SIGNATURE: zero evictions
DtoD:     0 MB  -- single GPU
End-to-end: 530.8 ms  (+ 231 ms CPU batch generation)
```

The zero DtoH is the critical signature of in-memory execution. The UVM driver prefetched the full 1,280 MB (678 MB table + 640 MB input) before the kernel launched, resolved each page fault once as a cold miss, and never evicted a page. Every subsequent access was a local VRAM hit. This is the **theoretical ceiling**: 354.7 M Ops/s.

3.3 The Oversubscription Cliff

Performance collapse is not gradual. It is a discrete cliff:

Scenario	Throughput	vs. Ceiling	Notes
In-memory (all fits)	354.7 M Ops/s	1×	Zero DtoH; 0 evictions
40 MB spill (<0.5%)	∼40 M Ops/s	8.9× slower	SM drainage begins
80M ops, 9× pressure	2.8 M Ops/s	128× slower	Full fault-migrate-replay
85M ops	OOM / killed	—	VRAM exhausted

Table 5: Single-GPU performance cliff under increasing oversubscription.

The transition at 0.5% spill (<1% fault probability) produces an 8.9× slowdown because the four fault mechanisms are *multiplicative*, not additive.

3.4 The Four Interlocking Fault Mechanisms

3.4.1 Mechanism 1: Synchronous Page Fault

When a warp's memory instruction targets a host-resident page, the GPU MMU fires a page fault interrupt. This is *synchronous*: the thread group cannot proceed until the CPU-side driver: (1) locates a free VRAM page (evicting a victim if necessary), (2) issues a PCIe DMA for the 4 KB page, (3) updates the GPU page table entry, (4) issues a TLB invalidation, (5) signals the GPU to allow replay. Total latency: ∼20–50 μs = ∼32,000–80,000 GPU cycles at 1.6 GHz (measured on Pascal; see Kim et al. ASPLOS 2020).

3.4.2 Mechanism 2: Warp Stall — All-or-Nothing Fault Decision

The warp-cooperative kernel assigns all 32 threads to one insert operation. Threads probe 32 *consecutive* slots (`pos + lane_id`, lanes 0–31), a 256-byte window fitting within a single 4 KB page. The fault decision is therefore *binary* per warp: either the page is resident (all 32 threads proceed) or it is spilled (all 32 threads stall). The fault probability equals the spill fraction:

$$P(\text{warp faults}) \approx \frac{\text{spilled pages}}{\text{total pages}} = \frac{40\text{ MB}/4\text{ KB}}{678\text{ MB}/4\text{ KB}} \approx 0.5\% \tag{9}$$

3.4.3 Mechanism 3: SM Drainage

A GTX 1080 SM supports up to 64 resident warps. At 0.5% fault probability, $64 \times 0.005 = 0.32$ warps fault per memory instruction cycle. Each fault stalls its warp for ∼50,000–80,000 cycles. The fault arrival rate (new faults per second) vs. the driver service rate (faults resolved per second) determines whether the ready-warp queue drains:

$$\text{Fault arrival rate} = \text{active warps} \times P(\text{fault}) \times \text{instructions/warp/cycle} \tag{10}$$

When faults arrive faster than the driver resolves them, the pool of ready warps shrinks monotonically until the SM pipeline empties. CPU profiling shows `cudaEventSynchronize` consuming >90% of CPU time during oversubscribed runs—the driver loop is saturated.

3.4.4 Mechanism 4: Fault-Migrate-Replay

After the driver migrates the requested page, the faulting warp's instruction is *replayed from scratch*. Replay itself costs additional cycles. Worse, migration of the new page often requires *evicting* another VRAM-resident page—which was likely accessed recently by another warp, creating a future fault. With uniform random access to a 678 MB table in an 899 MB VRAM budget (419 MB deficit), every eviction is almost certainly a page that will be needed again soon. The system enters a *thrashing* state where migration rate equals fault rate and throughput approaches zero.

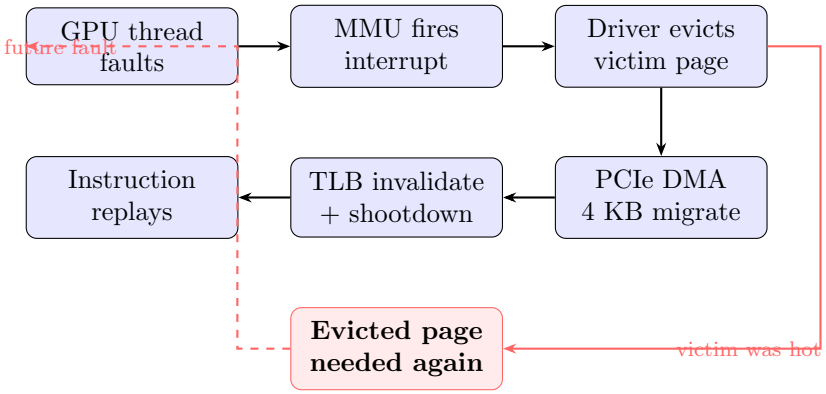


Figure 1: UVM fault-migrate-replay cycle. Evicting a hot victim page to service the fault creates a future fault on that same page, leading to thrashing.

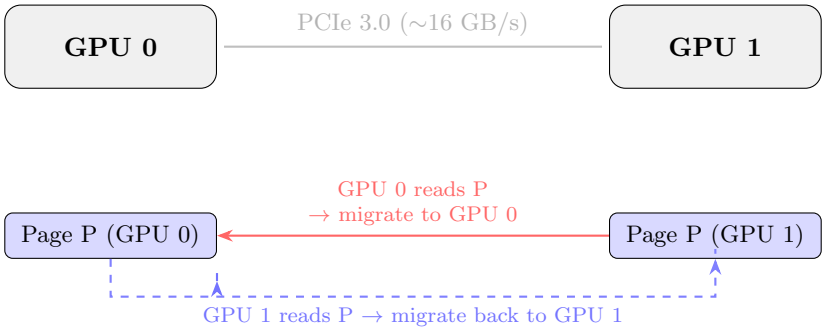
3.5 Naive Multi-GPU: The Ping-Pong Catastrophe

These two subsections establish why naïve multi-GPU approaches fail and motivate the radix solution in Section 6. CPU-level single-GPU schedulers are covered first in Section 5.

Running the unmodified kernel on both GPUs (each accessing the full shared hash table) collapsed throughput to 17.011 M Ops/s with 24.4 GB of DtoD traffic:

$$\text{Traffic amplification} = \frac{24.4 \text{ GB DtoD}}{640 \text{ MB workload}} = 38.1\times \tag{11}$$

The mechanism: GPU 0's thread accesses a slot whose page resides on GPU 1 → UVM migrates 4 KB GPU 1 to GPU 0. Moments later, GPU 1's thread accesses the same slot → UVM migrates 4 KB back to GPU 1. This bouncing persists for the duration of the workload, saturating the PCIe bus with redundant migrations.



Result: every shared page bounces continuously — 38× traffic amplification

Figure 2: Naive multi-GPU ping-pong. GPU 0 migrates a page from GPU 1; GPU 1 migrates it back. Pages bounce continuously over PCIe, generating 38× amplification.

The concurrent launch setup that creates this problem:

Listing 3: Naive multi-GPU: both GPUs launched on same UVM table.

```
// Enable P2P so UVM can migrate pages between GPUs
cudaDeviceEnablePeerAccess(1, 0);  // GPU 0 -> GPU 1
cudaDeviceEnablePeerAccess(0, 1);  // GPU 1 -> GPU 0

// Launch on GPU 0 (40M ops, same shared hash table gt)
cudaSetDevice(0);
batch_insert_gpu_kernel<<<grid, block, 0, streams[0]>>>(
    gt, gpu0_batch, gpu0_count, capacity, ...);

// Launch on GPU 1 (40M ops, same shared hash table gt)
cudaSetDevice(1);
batch_insert_gpu_kernel<<<grid, block, 0, streams[1]>>>(
    gt, gpu1_batch, gpu1_count, capacity, ...);

// Both kernels run concurrently -- every cross-GPU slot access
// triggers a 4KB page migration over PCIe
cudaStreamSynchronize(streams[0]);
cudaStreamSynchronize(streams[1]);
```

3.6 Hinted Multi-GPU: The Latency Trap

Using `cudaMemAdviseSetAccessedBy` on both GPUs establishes P2P page mappings without migration. DtoD traffic dropped to zero; throughput improved to 46.58 M Ops/s—but remained 7.6× below the in-memory ceiling.

Listing 4: Hinted multi-GPU: partition hints eliminating page migration.

```
void apply_partition_hints(KeyValue *gt, uint64_t capacity) {
  uint64_t half = capacity / 2;
  size_t half_bytes = half * sizeof(KeyValue);

  // Pin each half to its owning GPU
  cudaMemAdvise(gt,        half_bytes, cudaMemAdviseSetPreferredLocation
      , 0);
  cudaMemAdvise(gt + half, half_bytes, cudaMemAdviseSetPreferredLocation
      , 1);

  // Allow BOTH GPUs to read either half via Zero-Copy (no migration)
  cudaMemAdvise(gt,        half_bytes, cudaMemAdviseSetAccessedBy, 1);
  cudaMemAdvise(gt + half, half_bytes, cudaMemAdviseSetAccessedBy, 0);

  // Prefetch each half to its owning GPU before kernel launch
  cudaMemPrefetchAsync(gt,        half_bytes, 0);
  cudaMemPrefetchAsync(gt + half, half_bytes, 1);
  cudaSetDevice(0); cudaDeviceSynchronize();
  cudaSetDevice(1); cudaDeviceSynchronize();
}
```

Zero-Copy eliminates migration but every cross-partition probe pays 1–2 μs PCIe latency per access.

The fix introduced a new problem: *Zero-Copy reads*. Every cross-GPU slot access executes as a PCIe load instruction, incurring 1–2 μs per probe. An insert with 5 collision probes requires 5 sequential PCIe round-trips ($\sim$10 μs total). The SM drains for the same reason as the page-fault case, just with steady-state latency rather than discrete fault events.

This proved the fundamental constraint: **cross-GPU accesses cannot be managed in a random-access workload. They must be eliminated.**

Both multi-GPU baseline strategies failed because they still required *every GPU to access every partition*. Before discovering how to eliminate cross-GPU accesses entirely, we explored whether CPU-level scheduling on a single GPU could recover performance. Section 5 documents that exploration. Section 6 then delivers the root-cause fix.

4 The Hash Table Kernel: Design and Implementation

Presentation: Act I — The Foundation (slides s4a, s4b, s4c)

4.1 Data Structure: Open-Addressed Flat Array

The hash table is a single flat `KeyValue[]` array allocated with `cudaMallocManaged`:

Listing 5: Hash table data structure.

```
struct key_value_s {
    uint32_t key;    // 4 bytes; EMPTY_KEY = 0 (sentinel)
    uint32_t value;  // 4 bytes
};  // 8 bytes total, one slot

```

```
// Hash table allocation -- globally accessible from any GPU
cudaMallocManaged(&ht->table,
    capacity * sizeof(key_value_s),
    cudaMemAttachGlobal);
// capacity = 89,008,207 for 80M inserts at 90% load factor
```

There are no linked lists, no pointers, no buckets in the storage structure. Collisions are resolved by *open addressing with warp-stride-32 probing*: on a collision, the entire warp advances 32 slots to the next candidate window. This stride ensures the warp always probes a contiguous 256-byte block, fitting within one 4 KB UVM page.

4.2 MurmurHash: Why Not Simple Modulo

The project uses an **integer finalizer hash** [8], borrowed from the WarpCore implementation. This is commonly referred to as a "MurmurHash-inspired" or Thomas Wang-style integer mix—distinct from MurmurHash3's official `fmix32` finalizer (which uses constants `0x85ebca6b`/`0xc2b2ae35`). The alternative—Knuth multiplicative ("Fibonacci") hashing—was considered but rejected:

Listing 6: Integer finalizer hash used in all kernels (WarpCore variant).

```
// Integer finalizer (Thomas Wang / WarpCore variant)
// Note: NOT MurmurHash3 fmix32; uses constant 0x45d9f3b
__device__ __host__
uint32_t murmur_mix(uint32_t key) {
    key ^= key >> 16;
    key *= 0x45d9f3b;
    key ^= key >> 16;
    return key;
}

// Slot computation
uint64_t pos = (uint64_t)murmur_mix(key) % capacity;
```

Why this hash over Fibonacci hash:

- **Full avalanche**: every input bit affects every output bit. Fibonacci hashing ($k \times 2{,}654{,}435{,}761$) produces correlated outputs for structurally similar inputs (sequential keys, timestamps, IDs).
- **Radix scheduler correctness**: the routing histogram uses the same hash function to determine GPU ownership. If the hash function has poor bit distribution in the upper bits, the 50/50 partition split becomes skewed. The WarpCore integer mix's full mixing ensures near-perfect 40,001,193 / 39,998,807 split (measured).
- **Cost**: this hash requires ∼5 operations (2 XOR, 2 shift, 1 multiply) vs. ∼2 for Fibonacci. At 50,000+ cycles per page fault, this 3-cycle difference is immeasurable.

4.3 Warp-Cooperative Insert Protocol

The kernel assigns **one 32-thread warp per insert operation**. This design decision drives all subsequent behavior:

Listing 7: Warp-cooperative insert kernel (core protocol).

```
__global__ void batch_insert_gpu_kernel(
    key_value_s* __restrict__ ht,   // hash table
    key_value_s* __restrict__ ops,  // input buffer
    size_t capacity, size_t num_ops,
    uint32_t offset, uint32_t num_buckets)
{
    // Each warp handles one operation
    int warp_id  = (blockIdx.x * blockDim.x + threadIdx.x) / 32;
    int lane_id  = threadIdx.x % 32;
    if (warp_id >= num_ops) return;

    uint32_t key = ops[warp_id + offset].key;
    uint32_t val = ops[warp_id + offset].value;

    // Step 1: MurmurHash -> starting slot
    uint64_t pos = (uint64_t)murmur_mix(key) % capacity;

    // Step 2: Warp-cooperative linear probe
    while (true) {
        // All 32 lanes probe 32 consecutive slots (256 bytes = 1 page
           region)
        uint32_t existing = ht[pos + lane_id].key;

        // Step 3: Find empty slots via ballot
        unsigned empty_mask = __ballot_sync(
            0xFFFFFFFF, existing == EMPTY_KEY);

        if (empty_mask == 0) {
            pos = (pos + 32) % capacity;  // stride-32 probe
            continue;
        }

        // Step 4: Elect lowest-lane empty slot as leader
        int leader = __ffs(empty_mask) - 1;

        // Step 5: Leader atomically claims the slot
        uint32_t old = UINT32_MAX;
        if (lane_id == leader) {
            old = atomicCAS(&ht[pos + leader].key,
                            EMPTY_KEY, key);
        }

        // Step 6: Broadcast result -- zero memory traffic
        old = __shfl_sync(0xFFFFFFFF, old, leader);

        if (old == EMPTY_KEY) {
            // Claimed successfully; write value
            if (lane_id == leader)
                ht[pos + leader].value = val;
            break;
        }
```

```
            // Another warp claimed it; advance and retry
            pos = (pos + 32) % capacity;
        }
    }
```

4.3.1 Why Assign 32 Threads to One Insert

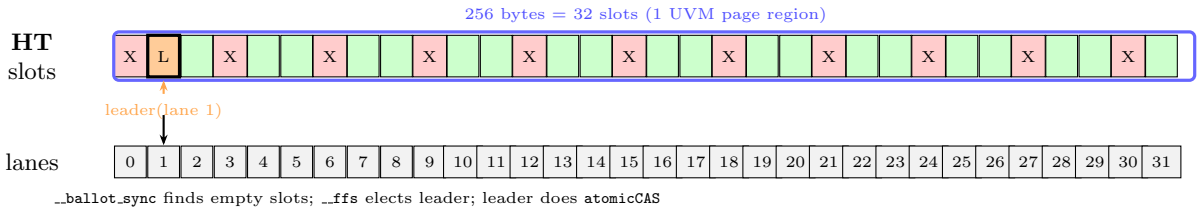


Figure 3: Warp-cooperative insert: 32 threads probe 32 consecutive slots (one 256-byte window). `__ballot_sync` identifies empty slots, the lowest-lane leader atomically claims one, and `__shfl_sync` broadcasts the result at zero memory cost.

- **Zero-cost coordination**: threads communicate via `__shfl_sync` through the SM's register file, with zero memory traffic. Broadcasting the leader's result to all 32 threads costs ∼1 cycle.
- **Parallelism within probing**: all 32 threads check 32 slots simultaneously, multiplying probe throughput.
- **Page fault alignment**: all 32 threads probe the same 4 KB page. The fault decision is all-or-nothing, making fault probability analysis tractable (Section 3.4).
- **Hardware efficiency**: 80M inserts × 32 threads/insert = 2.56 billion threads, exactly filling the SM pipeline at the GTX 1080's maximum occupancy.

4.4 Kernel Launch Configuration

Listing 8: Kernel launch geometry.

```
// Grid sizing: SDIV(N, 16) blocks x 512 threads
// 512 threads/block = 16 warps/block
// Each warp handles 1 operation
// Total warps = ceil(N / 16) * 16 >= N

dim3 grid(SDIV(num_ops, 16), 1, 1);
dim3 block(512, 1, 1);
batch_insert_gpu_kernel<<<grid, block>>>(ht, ops,
    capacity, num_ops, offset, num_buckets);
```

For 80M operations: $\lceil 80{,}000{,}000/16 \rceil = 5{,}000{,}000$ blocks, each with 512 threads = 5,000,000 × 1 × 1 grid configuration (verified in Nsight kernel summary).

4.5 CUDA Memory Allocation Strategy

The project uses two distinct allocation strategies with different performance characteristics:

Listing 9: Allocation strategy: UVM vs device-local.

```
// UVM: hash table, input buffers, dummy (all cross-device shared)
cudaMallocManaged(&ht->table,
    capacity * sizeof(key_value_s), cudaMemAttachGlobal);
cudaMallocManaged(&ops,
    N * sizeof(key_value_s), cudaMemAttachGlobal);
cudaMallocManaged(&dummy, 7293ULL*1024*1024,
    cudaMemAttachGlobal);  // oversubscription

// Device-local: histogram, sort buffers (never cross PCIe)
cudaMalloc(&d_histogram[d],
    NUM_RADIX_BUCKETS * sizeof(uint32_t));
cudaMalloc(&d_offsets[d],
    NUM_RADIX_BUCKETS * sizeof(uint32_t));
cudaMalloc(&d_sorted[d],
    N_per_gpu * sizeof(key_value_s));
```

The discipline: histogram arrays, radix sort buffers, and score trackers always use `cudaMalloc` (device-local) because they are private to one GPU and must never trigger UVM migrations.

5 CPU-Level Schedulers: Exploring the Scheduling Axis

Presentation: Act II — CPU Schedulers (slides sCPU1–sCPU6)

With the single-GPU oversubscription cliff established (2.77 M Ops/s, 128× collapse) and naïve multi-GPU approaches failing due to page thrashing, the natural question is: *can smarter CPU-side scheduling recover performance without changing the data layout?*

This section systematically explores six CPU-level strategies—pipelining, double-buffering, adaptive batch sizing, multistream concurrency, work-stealing, and intelligent prefetching—each on a **single GTX 1080** with **89% VRAM reserved**. The answer, established empirically across all six strategies, is **no**: scheduling alone cannot fix a capacity deficit. The best single-GPU result (Deep Pipeline, 80.5 M Ops/s) still leaves 77% of the in-memory ceiling unreached because the 419 MB VRAM deficit persists regardless of dispatch order.

All schedulers use the same hardware configuration: All schedulers run on a **single GTX 1080** with **89% VRAM reserved** (7,293 MB dummy, ∼899 MB free), oversubscription ratio 9.1×. Branch names correspond to Git branches in the project repository.

5.1 Configuration

- GPU: single GTX 1080 (8,192 MB VRAM)
- **Dummy allocation: 7,293 MB = 89% of VRAM reserved**; leaving ∼899 MB free
- Oversubscription ratio: 8,192 MB total/899 MB free ≈ 9.1×
- Hash table: 89,008,207 slots (≈678 MB)
- Input: 80M key-value pairs (640 MB)

- Compile flags: `-std=c++17 -arch=sm_61 -O3 -DUVM_MEM_ADVISE_SA -DUVM_PREFETCH_HINT`

Branch	Strategy	Throughput (M Ops/s)	Wall-Clock (ms)	HtoD (MB)	DtoH (MB)
`scheduler-deep-pipeline`	Depth-4 ring, 5M batches	**80.532**	993	640	630
`scheduler-workstealing`	16×5M batches	63.958	1,251	640	685
`scheduler-multistream`	4 streams	29.647	2,698	640	629
`scheduler-pipeline`	Double-buffer 80M	23.855	3,354	640	706
`scheduler-adaptive`	Feedback control	23.841	3,356	640	706
`scheduler-prefetch`	CPU prediction	2.123	37,685	54,944	54,965

Table 6: All CPU-level scheduler results, single GPU, oversubscription 9.1×, 80M inserts.

5.2 Work-Stealing: 63.958 M Ops/s

Branch: `scheduler-workstealing` · **Compile**: `-gbs=5000000`

5.2.1 Hypothesis

Divide the 80M-op batch into 16 smaller batches of 5M each. If the GPU becomes saturated, overflow operations are stolen by CPU worker threads operating on a separate CPU-side hash table.

5.2.2 Why It Works: Key Buffer Shrinkage

The standard analysis claims each 5M batch "touches only 42 MB of the table (6.25%)"—this is incorrect for random access. 5M keys with the integer finalizer hash still scatter across essentially all 174,000 pages of the 678 MB table. The actual mechanism is *key buffer shrinkage*:

$$\underbrace{640 \text{ MB}}_{\text{80M pairs}} + 678 \text{ MB} = 1{,}318 \text{ MB} \quad \text{(419 MB deficit, continuous eviction)} \tag{12}$$

$$\underbrace{40 \text{ MB}}_{\text{5M pairs}} + 678 \text{ MB} = 718 \text{ MB} \quad \text{(181 MB surplus, table stays resident)} \tag{13}$$

Reducing the batch to 5M shrinks the key buffer from 640 MB to 40 MB, bringing total working set below the 899 MB VRAM budget. The entire hash table can now remain resident throughout the batch.

5.2.3 The Naming Irony

The CPU work-stealing mechanism processed **zero operations**:

Listing 10: Work-stealing runtime output.

```
=== Work Stealing Results ===
Total batches: 16        Total operations: 80,000,000
Total GPU kernel time:   549.648 ms
Total wall-clock time: 1,250.82 ms
Overall throughput:       63.958 M Ops/s

```

```
Work Stealing Stats:
  GPU operations: 80,000,000
  CPU operations:          0    <-- fallback never triggered
  Queue full events:       0
  CPU work ratio:          0%
```

Each 5M batch completed in ∼34 ms—fast enough that the GPU queue never backed up. The CPU offload was armed but irrelevant. **Batch decomposition alone was the win**.

5.2.4 Nsight Memory Traffic

Listing 11: Nsight gpumemsizesum – Work-Stealing.

```
HtoD: 640.025 MB / 320 transfers  (avg 2.000 MB)
DtoH: 685.044 MB / 3,419 transfers (avg 0.200 MB)
```

The 685 MB DtoH in 3,419 small transfers (avg 200 KB) shows incremental per-batch evictions: each batch still faults on some hash table pages (since 718 MB ≈ 899 MB budget with comfortable slack), but evictions are spread across 16 batches in small bursts rather than one catastrophic cascade.

5.3 Deep Pipeline: 80.532 M Ops/s

Branch: `scheduler-deep-pipeline` · **Compile**: `-Xcompiler -pthread`

5.3.1 Motivation

Work-Stealing ran batches sequentially: fill buffer on CPU → launch GPU kernel → wait for GPU → fill next buffer. During each CPU fill phase (∼33 ms), the GPU was completely idle. Deep Pipeline eliminates this waste.

Listing 12: Deep pipeline ring-buffer data structures.

```
struct DeepPipelineBuffer {
  KeyValue        *data;       // UVM buffer (5M slots = 40 MB)
  cudaStream_t     stream;     // Dedicated CUDA stream
  cudaEvent_t      done_event; // Signals GPU completion
  uint64_t         count;
  std::atomic<bool> ready;     // CPU has filled this buffer
  std::atomic<bool> in_flight; // GPU is consuming this buffer
};

struct DeepPipelineState {
  DeepPipelineBuffer *buffers;          // Ring of N=4 buffers
  std::atomic<uint32_t> cpu_write_idx;  // CPU writes here next
  std::atomic<uint32_t> gpu_read_idx;   // GPU reads from here
  std::atomic<uint64_t> ops_prepared;
  std::atomic<uint64_t> ops_completed;
  std::atomic<uint32_t> cpu_wait_count; // CPU blocked on full ring
  std::atomic<uint32_t> gpu_wait_count; // GPU blocked on empty ring
};
```

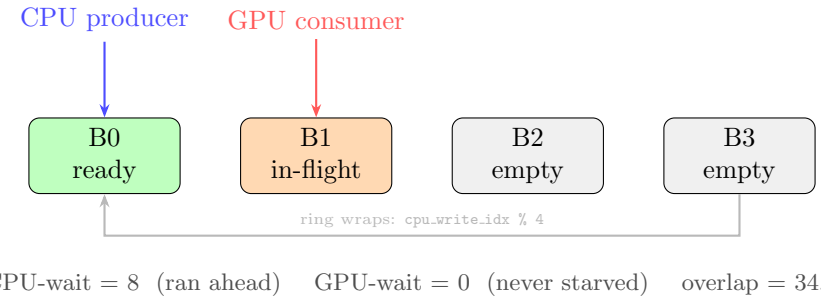


Figure 4: Deep Pipeline depth-4 ring buffer. The CPU producer fills empty slots while the GPU consumer launches kernels from ready slots. At 97% CPU/GPU time ratio, the CPU runs 1–4 batches ahead with zero GPU starvation.

5.3.2 The CPU/GPU Ratio Insight

Pipelining only delivers overlap when CPU and GPU times are comparable. The 80M-batch Pipeline had a 7% ratio (CPU 206 ms, GPU 3,013 ms), making overlap irrelevant. Work-Stealing's 5M batches changed everything:

Configuration	CPU fill	GPU kernel	Ratio
Pipeline (80M batch)	206 ms	3,013 ms	6.8%
Deep Pipeline (5M batch)	∼33 ms	∼34 ms	**97%**

Table 7: CPU/GPU time ratio determines pipeline effectiveness.

At 97% ratio, a depth-4 ring buffer lets the CPU run up to 4 batches ahead:

Listing 13: Depth-4 ring buffer implementation.

```
// CPU producer thread (dedicated pthread)
while (ops_prepared < total_ops) {
    int slot = cpu_write_idx % 4;
    while (ring[slot].in_flight) sched_yield();
    fill_buffer(ring[slot], src, start, batch_size);
    ring[slot].ready = true;
    cpu_write_idx++;
}

// GPU consumer (main thread)
while (ops_done < total_ops) {
    int slot = gpu_read_idx % 4;
    while (!ring[slot].ready) sched_yield();
    ring[slot].in_flight = true;
    cudaLaunchKernel(insert, ring[slot]);
    cudaEventSynchronize(ring[slot].done_event);
    ring[slot].in_flight = false;
    gpu_read_idx++;
}
// Buffer memory: 4 x 5M x 8B = 152 MB total
```

5.3.3 Measured Results

Listing 14: Deep Pipeline runtime output.

```
=== Deep Pipeline Results ===
Total batches: 16        Total operations: 80,000,000
Batch size: 5,000,000 ops    Number of buffers: 4

--- Timing ---
Total CPU prep time:  530.389 ms
Total wall-clock:     993.395 ms
Throughput:           80.5319 M Ops/s

--- Pipeline Efficiency ---
CPU wait events (buffer in-flight):  8   (CPU ran ahead -- ideal)
GPU wait events (buffer not ready):  0   (GPU never starved -- ideal)
Estimated overlap benefit:      34.8074%
Improvement vs Work-Stealing:   25.8311%
```

Listing 15: Nsight gpumemsizesum – Deep Pipeline.

```
HtoD: 640.025 MB / 320 transfers  (avg 2.000 MB)
DtoH: 630.051 MB / 2,754 transfers (avg 0.229 MB)
```

The 8 CPU-wait events (vs. 0 GPU-wait events) confirm the ideal operating regime: the CPU ran ahead on exactly half the 16 batch transitions, the GPU never stalled for data.

5.3.4 Why It Still Cannot Reach the Ceiling

DtoH = 630 MB confirms that page faults still occur per batch. The 718 MB total working set (40 MB key buffer + 678 MB table) sits only 181 MB below the 899 MB VRAM budget. GPU runtime state, kernel metadata, and stack frames consume that margin. The hash table is still partially paged out on every batch transition. Deep Pipeline eliminates all CPU-side waste but cannot fix the fundamental capacity deficit.

5.4 Multistream: 29.647 M Ops/s

Branch: `scheduler-multistream`

Listing 16: Multistream: StreamContext struct and input partition.

```
struct StreamContext {
  cudaStream_t  stream;
  cudaEvent_t   start_event, stop_event;
  KeyValue     *buffer;  // UVM buffer for this stream's partition
  uint64_t      count;
};

// Partition input evenly across N streams (CPU memcpy, no kernel)
void partition_input(KeyValue *src, uint64_t total_ops,
                     std::vector<StreamContext> &streams) {
  uint64_t per_stream = total_ops / streams.size();
  uint64_t offset = 0;
```

```
  for (auto &ctx : streams) {
    ctx.count = per_stream;
    memcpy(ctx.buffer, src + offset, per_stream * sizeof(KeyValue));
    offset += per_stream;
  }
}

// Launch all N stream kernels asynchronously
void launch_all_streams(std::vector<StreamContext> &streams,
                        KeyValue *ht, uint64_t capacity) {
  for (auto &ctx : streams) {
    cudaMemPrefetchAsync(ctx.buffer,
                         ctx.count * sizeof(KeyValue), 0, ctx.stream);
    cudaEventRecord(ctx.start_event, ctx.stream);
    batch_insert_gpu_kernel<<<SDIV(ctx.count<<5, 512), 512,
                              0, ctx.stream>>>(ht, ctx.buffer,
                              ctx.count, capacity, ...);
    cudaEventRecord(ctx.stop_event, ctx.stream);
  }
  for (auto &ctx : streams) cudaStreamSynchronize(ctx.stream);
}
```

5.4.1 In-Memory Rationale

Without oversubscription, multistream achieves 305.6 M Ops/s—the best single-GPU in-memory technique. The mechanism: multiple independent CUDA streams enable *concurrent kernel execution*, allowing thread blocks from different kernels to be co-resident on the same SM simultaneously. This increases the total number of warps available to each SM's warp schedulers. When any warp stalls on VRAM latency (∼600 ns), the SM's warp scheduler—which is stream-agnostic and switches between warps, not streams—selects any other ready warp from its pool (which may originate from any co-resident block of any kernel). With more co-resident blocks from multiple streams, the pool is larger and more diverse, improving latency hiding and bandwidth utilization.

Note: CUDA streams are software command queues with no 1:1 hardware mapping to SMs. There is no "one stream per SM" relationship. A single SM executes thread blocks from multiple streams simultaneously when resource capacity (registers, shared memory, warp slots) permits.

5.4.2 Two Failures Under Oversubscription

Hardware failure: The GTX 1080 (sm_61) reports `concurrentKernels = 1`, which is a *boolean capability flag* meaning the device is *capable* of concurrent kernel execution—not a limit of one kernel. The hardware ceiling for sm_6.1 is 32 resident grids (concurrent kernels). However, in practice all 4 stream kernels serialize because each kernel already occupies the full GPU: with 80 M operations dispatched to a single hash table, the first kernel's thread blocks saturate all 20 SMs, consuming all warp slots (64 per SM × 20 SMs = 1,280 warps). There is no SM resource headroom for a second kernel's blocks to be co-scheduled. The streams are logically concurrent but physically serialize due to occupancy saturation:

Listing 17: Runtime output confirming effective serialization.

```
GPU: NVIDIA GeForce GTX 1080
```

```
concurrentKernels (capability flag): 1   <-- boolean: "yes, capable of
    concurrent kernels"
Max resident grids (sm_6.1 hardware limit): 32
Observed behavior: all 4 streams serialize  <-- occupancy-driven, not a
    hardware limit of 1
```

Memory failure: Because streams serialize, each stream evicts the previous stream's pages:

Listing 18: Per-stream timing showing monotonic degradation.

```
=== Multi-Stream Results ===
Partition time: 201.445 ms
Kernel execution time (max): 2555.48 ms
Total wall-clock:             2698.38 ms
Throughput:                   29.6474 M Ops/s

Per-stream timing:
  Stream 0:   777.145 ms (25.7352 M Ops/s)  -- cache warm
  Stream 1: 1,330.34  ms (15.0338 M Ops/s)  -- S0 evicted pages
  Stream 2: 1,906.04  ms (10.4930 M Ops/s)  -- 2 streams evicted
  Stream 3: 2,555.48  ms ( 7.8263 M Ops/s)  -- 3 streams evicted
```

Listing 19: Nsight gpumemsizesum – Multistream.

```
HtoD: 640.008 MB / 308 transfers (avg 2.078 MB)
DtoH: 628.802 MB / 301 transfers (avg 2.089 MB)
```

DtoH ≈ HtoD: every page migrated in was evicted out. The PCIe bus runs a zero-sum page shuffle. The 10.3× degradation from in-memory (305.6) to oversubscribed (29.6) is the most dramatic inversion in the experiment set.

5.5 Pipeline: 23.855 M Ops/s

Branch: `scheduler-pipeline`

Listing 20: Pipeline runtime output.

```
=== Pipeline Results ===
Total batches: 1         Total operations: 80,000,000
Total CPU prep time:   206.238 ms
Total wall-clock:    3,353.55 ms
Throughput:           23.8553 M Ops/s
Estimated overlap benefit: 5.79354%
```

Listing 21: Nsight gpumemsizesum – Pipeline.

```
HtoD: 640.000 MB / 306 transfers (avg 2.092 MB)
DtoH: 705.778 MB / 337 transfers (avg 2.094 MB)
```

The 5.79% overlap benefit is statistically marginal: GPU kernel dominates 3,013 ms of 3,354 ms total (89.8% GPU-bound). The CPU prepares its buffer in 206 ms then idles for 2,807 ms. Double-buffering helps when neither side dominates; here the GPU is 14.6× slower than the CPU.

5.6 Adaptive Batch Sizing: 23.841 M Ops/s

Branch: `scheduler-adaptive`

5.6.1 Design

A throughput-based feedback controller (inspired by TCP slow-start) dynamically adjusts batch size: launch a batch, measure throughput, increase size if throughput improved, decrease if it degraded. The hypothesis: the controller will converge to the optimal batch size that minimizes fault frequency.

Listing 22: AdaptiveController: TCP slow-start feedback logic.

```
// Constants (from adaptive_controller.h)
constexpr float THROUGHPUT_DROP_THRESHOLD  = 0.80f; // 20% drop -> shrink
constexpr float THROUGHPUT_STABLE_THRESHOLD= 0.95f; // < 5% drop -> stable
constexpr float INCREASE_FACTOR            = 1.20f; // grow 20% when stable
constexpr float DECREASE_FACTOR            = 0.50f; // halve when pressure

void AdaptiveController::record_batch(uint64_t size, float ms) {
  float tput = (size / ms) * 1000.0f / 1e6f;  // M Ops/s

  if (history.size() >= 2) {
    float prev  = history.back().throughput_mops;
    float ratio = tput / prev;

    if (ratio < THROUGHPUT_DROP_THRESHOLD && size >= history.back().batch_size) {
      // Memory pressure detected: halve batch size (exponential backoff)
      current_batch_size = std::max(MIN_BATCH_SIZE,
          (uint64_t)(current_batch_size * DECREASE_FACTOR));
      consecutive_drops++;  consecutive_stable = 0;
    } else if (ratio >= THROUGHPUT_STABLE_THRESHOLD) {
      // Stable: grow batch by 20% (slow-start increase)
      current_batch_size = std::min(MAX_BATCH_SIZE,
          (uint64_t)(current_batch_size * INCREASE_FACTOR));
      consecutive_stable++;  consecutive_drops = 0;
    }
  }
  // ...history tracking and peak bookkeeping omitted
}
```

5.6.2 Failure: Initialization Bug

Listing 23: Adaptive scheduler runtime – only 1 batch processed.

```
=== Adaptive Batch Sizing Results ===
Total batches:     1
Total operations:  80,000,000
```

```
Total kernel time: 3,026.68 ms
Wall-clock:        3,355.53 ms
Throughput:        23.8412 M Ops/s

Adaptive Controller Stats:
  Initial batch size: 100,000,000
  Current batch size: 100,000,000
  Optimal batch size:  80,000,000  (capped to actual ops)
  Batch History:
    [0] size=80,000,000 time=3026.68ms throughput=26.43 M Ops/s
```

The controller initialized with `initial_batch_size = 100,000,000`—larger than the total 80M ops. It processed one batch (capped to 80M), had no comparison point, and the feedback loop never ran.

5.6.3 Correct Behavior (1M Initial Batch)

A separate run initialized at 1M batch size converged to:

- Optimal batch: **1.0–1.44M ops** (1.25–1.8% of workload per launch)
- End-to-end throughput: **75.2 M Ops/s**
- Peak per-batch throughput: **215.2 M Ops/s** (61% of in-memory ceiling!)

The 215.2 M/s per-batch peak is possible because a 1M-op batch uses only 8 MB of key buffer, leaving 792 MB for the hash table—ample VRAM for the entire table with no evictions during that batch.

5.7 Intelligent Prefetch: 2.123 M Ops/s

Branch: `scheduler-prefetch`

5.7.1 Research Foundation

Based on SAFE (IEEE CAL 2025) [5] and MSched (arXiv 2025) [6]. Both papers demonstrate that GPU access patterns can be predicted from kernel arguments to pre-warm page caches before kernel launch.

5.7.2 Implementation

Listing 24: CPU-side prediction loop (the fatal flaw).

```
// Predict which pages the kernel will access
for (int i = 0; i < num_ops; i++) {
    for (int depth = 0; depth < 8; depth++) {
        uint64_t slot = murmur_mix(ops[i].key) % capacity
                        + depth * 32;
        // FATAL: table is cudaMallocManaged
        // This CPU read migrates the 4KB page GPU->CPU
        if (table[slot].key != EMPTY_KEY) continue;
        hot_pages.insert(slot / 512);
    }
```

```
}
// Issue prefetch for predicted pages
cudaMemPrefetchAsync(page_addr, PAGE_SIZE, gpu_id, stream);
```

5.7.3 The Hidden UVM Rule

The CPU-Reads-UVM Anti-Pattern

`cudaMallocManaged` memory has a *current owner*. The CPU's MMU cannot directly access GPU VRAM. Any CPU read of a managed address whose page resides in GPU VRAM causes the driver to migrate that 4 KB page from GPU VRAM to host RAM before returning the value. This is a hardware constraint, not a policy choice.

$$80\text{M keys} \times 8 \text{ probe depths} = 640\text{M CPU reads} \Rightarrow 640\text{M page migrations (GPU} \rightarrow \text{CPU)} \quad (14)$$

The entire 678 MB hash table is evicted to host RAM before the kernel launches. The kernel refaults every page from scratch.

5.7.4 Nsight Evidence

Listing 25: Nsight output – Intelligent Prefetch catastrophe.

```
=== Intelligent Prefetch Results ===
Timing breakdown:
  Prediction time:      12,028.9 ms  (640M CPU reads)
  Prefetch issue time:       0.021 ms (173,845 requests queued)
  Kernel time:          11,757.7 ms  (cold table -- all faults)
  Total wall-clock:     37,684.7 ms
Throughput: 2.12288 M Ops/s

Prefetch Predictor Stats:
  Pages tracked: 173,845  (all pages classified hot)
  Pages actually prefetched: 1  (queue overwhelmed, never drained)

Nsight gpumemsizesum:
  HtoD: 54,944.489 MB / 1,244,282 transfers (avg 0.044 MB)
  DtoH: 54,965.395 MB /    26,297 transfers (avg 2.090 MB)
```

The diagnostic signature: 0.044 MB average HtoD transfer (vs. 2.09 MB for all other schedulers). The UVM fault handler resolves individual instruction-level faults one at a time—4,000× more transfers at 1/47th the size. Total PCIe traffic: **110 GB for a 640 MB workload** (172× amplification).

Insight: All CPU-level schedulers address the wrong problem

Every strategy in this section operates on the *scheduling axis* (when and how batches are dispatched). None reduces the fundamental 419 MB VRAM deficit. The only schedulers that meaningfully outperform the baseline (Work-Stealing, Deep Pipeline) succeed because they accidentally reduce the key-buffer footprint—a capacity effect, not a scheduling effect. The

next section identifies and exploits this root cause directly.

6 Radix Routing: The Three-Phase Solution

Presentation: Act III — The Solution (slides s5–s11b)

The Root Cause Fix

CPU-level scheduling (Section 5) established that no dispatch strategy eliminates the 419 MB VRAM deficit. The only path forward is *data partitioning*: route each key to the GPU that owns its target hash table slot *before* any insert kernel executes. After routing, both the key and its destination page are local VRAM—zero cross-GPU accesses, zero page faults during insertion.

6.1 Architectural Inspiration: The Macro-Cache Analogy

The radix solution repurposes a cache optimization technique from DACHash (2021) [2] at the UVM level:

Level	Transfer Unit	Miss Penalty	Software Mitigation
L2 cache line	128 B	∼200 cycles	DACHash radix reordering
UVM page	4,096 B	∼50,000–80,000 cycles	**This project's radix routing**
Penalty ratio: ∼50,000 / 200 = 250×, so UVM payoff is ∼250× larger			

Table 8: The macro-cache analogy connecting DACHash to this work.

A UVM page is a 50,000–80,000-cycle "macro cache line." The same reordering that avoids L2 misses—group accesses by address range before execution—also avoids UVM faults, with ∼250× higher return on investment.

6.2 Core Insight: Partition Alignment

The fundamental error in naive multi-GPU execution is treating UVM as "shared memory." Every cross-GPU access pays a massive PCIe penalty. The solution:

Key Insight: Partition Alignment

Route each key to the GPU that owns its target hash table slot *before* any insert kernel executes. Guarantee: when a GPU thread performs an insert, both the key-value pair and the target table slot are in local VRAM. Zero cross-GPU accesses during execution.

The routing decision is pure arithmetic—no memory access required:

Listing 26: GPU ownership determination (pure computation).

```
uint64_t hash_pos = murmur_mix(key) % capacity;
int owner_gpu = (hash_pos < capacity / 2) ? 0 : 1;
// GPU 0 owns slots [0, 44,504,103)  -- 339 MB
```

```
// GPU 1 owns slots [44,504,103, 89,008,207)  -- 339 MB
```

6.3 Phase 1: GPU Routing (Histogram + Scatter)

Listing 27: Radix histogram kernel – two-level reduction.

```
__global__ void radix_histogram_kernel(
    key_value_s* keys, uint32_t* histogram,
    int N, size_t capacity)
{
    // Two-level reduction: shared memory first, then global
    __shared__ uint32_t local_hist[NUM_RADIX_BUCKETS];

    if (threadIdx.x < NUM_RADIX_BUCKETS)
        local_hist[threadIdx.x] = 0;
    __syncthreads();

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        uint64_t h = murmur_mix(keys[idx].key) % capacity;
        uint32_t bucket = (h < capacity/2) ? 0 : 1;
        atomicAdd(&local_hist[bucket], 1);
    }
    __syncthreads();

    if (threadIdx.x < NUM_RADIX_BUCKETS)
        atomicAdd(&histogram[threadIdx.x],
                  local_hist[threadIdx.x]);
}
```

After the histogram, a prefix-sum pass computes scatter offsets, and a scatter kernel reorders the input array into two contiguous partitions. Phase 1 measured timing: **128.1 ms** (Nsight).

Listing 28: Phase 1 scatter kernel – warp-level coalesced routing.

```
__global__ void gpu_routing_scatter_kernel(
    const KeyValue *input, uint64_t num_ops, uint64_t capacity,
    KeyValue *gpu0_out, KeyValue *gpu1_out,
    uint32_t *gpu0_counter, uint32_t *gpu1_counter)
{
  uint64_t gid = (uint64_t)blockIdx.x * blockDim.x + threadIdx.x;
  if (gid >= num_ops) return;

  int owner = (murmur_mix(input[gid].key) % capacity < capacity / 2) ? 0
      : 1;

  // Warp-level aggregation: elect one thread per warp per GPU partition
  // to do a single atomicAdd instead of N individual atomics
  uint32_t mask0 = __ballot_sync(0xFFFFFFFF, owner == 0);
  uint32_t mask1 = __ballot_sync(0xFFFFFFFF, owner == 1);
  uint32_t lane  = threadIdx.x % 32;

  if (owner == 0) {
```

```
    uint32_t warp_pos = __popc(mask0 & ((1u << lane) - 1));
    uint32_t base = 0;
    if (lane == __ffs(mask0) - 1)
      base = atomicAdd(gpu0_counter, __popc(mask0));
    base = __shfl_sync(mask0, base, __ffs(mask0) - 1);
    gpu0_out[base + warp_pos] = input[gid];
  } else {
    uint32_t warp_pos = __popc(mask1 & ((1u << lane) - 1));
    uint32_t base = 0;
    if (lane == __ffs(mask1) - 1)
      base = atomicAdd(gpu1_counter, __popc(mask1));
    base = __shfl_sync(mask1, base, __ffs(mask1) - 1);
    gpu1_out[base + warp_pos] = input[gid];
  }
}
```

The GPU-1 partition is then transferred via `cudaMemcpy(DeviceToDevice)`:

Listing 29: Bulk routing transfer – one DMA, minimum traffic.

```
// One coalesced bulk transfer: 320 MB in 154 transactions
cudaMemcpy(gpu1_ops, gpu0_gpu1_partition,
           gpu1_count * sizeof(key_value_s),
           cudaMemcpyDeviceToDevice);
// Measured: 320 MB / 154 transfers (avg 2.078 MB each) = 24.0 ms
```

6.4 Phase 2: Local Radix Partitioning

After routing, each GPU has ∼40M keys—all destined for its own 339 MB partition. But within that partition, the keys remain in random order, causing intra-partition cache thrashing. Phase 2 sub-divides each GPU's keys into B buckets (64 or 128) by the upper bits of the local hash position:

Listing 30: Local bucket assignment.

```
__device__ uint32_t local_bucket(
    uint32_t key, size_t capacity,
    int gpu_id, int num_buckets)
{
    uint64_t h = murmur_mix(key) % capacity;
    uint64_t local_pos = (gpu_id == 0)
        ? h
        : h - capacity / 2;
    return (local_pos * num_buckets) / (capacity / 2);
}
// Bucket b covers slots [b*(cap/2)/B, (b+1)*(cap/2)/B)
// For B=64: each bucket = ~5.3 MB of the 339 MB partition
// For B=128: each bucket = ~2.65 MB  <-- fits in 2MB L2!
```

Phase 2 measured timing: **110.8 ms**. No PCIe traffic—pure GPU-local sort.

Listing 31: Phase 2 local radix scatter – sorts each GPU's keys into B buckets.

```
__global__ void local_radix_scatter_kernel(
```

```
    const KeyValue *input, uint64_t num_ops, uint64_t capacity,
    KeyValue *output, const uint32_t *bucket_offsets,
    uint32_t *bucket_counters)
{
  uint64_t gid = (uint64_t)blockIdx.x * blockDim.x + threadIdx.x;
  if (gid >= num_ops) return;

  uint32_t bucket = get_local_bucket_id(input[gid].key, capacity);
  // One atomicAdd per thread to claim output position
  uint32_t pos = atomicAdd(&bucket_counters[bucket], 1)
                 + bucket_offsets[bucket];
  output[pos] = input[gid];
}
// get_local_bucket_id maps local hash position -> bucket [0, B)
// For B=128: each bucket covers ~2.65 MB -> fits in 2 MB L2 cache
```

6.4.1 Why B=128 Beats B=64

With $B = 128$, each bucket's working set (∼2.65 MB) fits within the GTX 1080's 2 MB L2 cache. The insertion kernel sees L2 hits instead of DRAM accesses, reducing memory latency by ∼3×:

Buckets	Per-bucket size	L2 fit?	Throughput	Wall-clock
64	∼5.3 MB	No (2.6× over L2)	174.672 M/s	458 ms
128	∼2.65 MB	**Yes**	**223.487 M/s**	**358 ms**

Table 9: Effect of bucket count on L2 cache fit and throughput.

6.5 Phase 3: Bucketed Insertion

Each GPU launches B sequential kernels, one per bucket. Within a bucket, all ∼625K keys target the same ∼5.3 MB (or 2.65 MB) region:

Listing 32: Bucketed insertion loop – zero cross-GPU traffic.

```
for (int b = 0; b < NUM_RADIX_BUCKETS; b++) {
    int start = offsets[b];
    int count = (b+1 < NUM_RADIX_BUCKETS)
        ? offsets[b+1] - offsets[b]
        : total - offsets[b];
    if (count == 0) continue;

    dim3 grid(SDIV(count, 16), 1, 1);
    dim3 block(512, 1, 1);
    batch_insert_gpu_kernel<<<grid, block, 0, stream>>>(
        ht, sorted_ops, capacity, count, start, b);
}
// DtoH = 0 MB for entire Phase 3 (Nsight verified)
```

6.5.1 Numeric Walkthrough

Consider 4 keys with hash positions on a small table:

Key	Hash pos	GPU	Local idx	Bucket
K1	100,500	0	100,500	19
K2	850,200	1	338,200	66
K3	100,650	0	100,650	19
K4	100,410	0	100,410	19

Table 10: Example key routing for a 1,024,000-slot table with 100 buckets per GPU.

Execution of Bucket 19 on GPU 0:

1. **K1** targets slot 100,500. Page fault: driver migrates 4 KB page covering slots 100,352–100,863 to GPU 0.
2. **K4** targets slot 100,410. **Zero fault**—same page, already resident.
3. **K3** targets slot 100,650. **Zero fault**—same page.

Result: 1 page fault for 3 inserts (vs. 3 faults in the naive case). At scale: 154 DtoD transfers vs. 38,000+ for naive.

6.6 UVM Advice and Prefetch Setup

Before Phase 1 begins, the driver applies partition hints and prefetches:

Listing 33: UVM partition setup in scheduler-radix.

```
// Pin each half of hash table to its owning GPU
cudaMemAdvise(ht->table, half_size,
    cudaMemAdviseSetPreferredLocation, 0);
cudaMemAdvise(ht->table + half_slots, half_size,
    cudaMemAdviseSetPreferredLocation, 1);

// Pre-load partitions into VRAM before kernel launch
cudaMemPrefetchAsync(ht->table, half_size, 0, stream);
cudaMemPrefetchAsync(ht->table + half_slots,
    half_size, 1, stream);

// Pre-load input to GPU 0 (before routing)
cudaMemPrefetchAsync(ops, N * sizeof(key_value_s),
    0, stream);  // 640 MB, 53.6 ms measured
```

`-DUVM_MEM_ADVISE_SA` compile flag enables all `cudaMemAdvise` calls. Without this flag, the driver uses default migrate-on-touch policy.

6.7 Full Measured Results: Radix 64 Buckets

Listing 34: Complete Nsight output for scheduler-radix (64 buckets, ovr=9.0).

```
GPU 0 operations: 40,001,193
GPU 1 operations: 39,998,807

[NOT TIMED] memcpy CPU->UVM:          211.753 ms
```

```
[NOT TIMED] cudaMemPrefetchAsync:       ~53.6 ms  (640 MB)
---
[TIMED] Phase 1 - Routing:              128.079 ms
[TIMED] Phase 2 - Local Radix:          110.803 ms
[TIMED] Phase 3 - GPU 0 insert:         104.769 ms -> 381.8 M Ops/s
[TIMED] Phase 3 - GPU 1 insert:         114.300 ms -> 350.0 M Ops/s
[TIMED] Wall-clock insert:              219.118 ms (max of both GPUs)
[TIMED] Total wall-clock:               458.001 ms
Combined throughput:                    174.672 M Ops/s
Radix overhead:                         52.16%

Nsight gpumemsizesum:
  HtoD: 640.000 MB / 306 transfers (avg 2.092 MB)
  DtoD: 319.996 MB / 154 transfers (avg 2.078 MB)
  DtoH:   0.000 MB  <-- zero evictions, table stays resident
```

The Critical Result

Per-GPU insertion throughput (GPU 0: 381.8 M/s, GPU 1: 350.0 M/s avg ≈ 365 M/s) **matches or exceeds** the single-GPU in-memory ceiling (354.7 M/s). After radix routing, the insertion phase is indistinguishable from having infinite VRAM.

6.8 Overhead Breakdown

Phase	Time (ms)	% Wall-Clock	Notes
Batch creation (CPU)	∼212	—	Not timed; overlaps some GPU setup
Input prefetch (HtoD)	∼54	—	Not timed
Phase 1: Routing	128.1	28.0%	Histogram + scatter + DtoD transfer
Phase 2: Local radix	110.8	24.2%	Local sort into 64 buckets per GPU
Phase 3: Insertion	219.1	47.8%	Parallel, both GPUs, zero PCIe
Total (timed)	**458.0**	**100%**	

Table 11: Phase-level timing for Radix 64 buckets (oversubscription 9.1×).

The insertion phase (47.8% of wall time) runs at compute-bound speed. The 52.2% overhead is the *one-time tax* for eliminating all page faults during insertion.

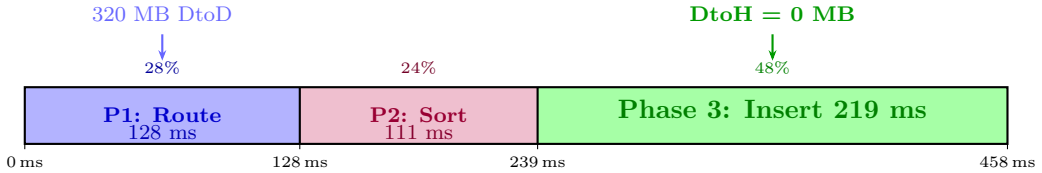


Figure 5: Radix 64 wall-clock timeline (458 ms total). Phase 3 insertion runs at near in-memory speed (DtoH = 0 MB). The 52% routing overhead is the one-time tax for eliminating all page faults.

6.9 Super-Radix: Pipelining the Routing Overhead

The `scheduler-super-radix` branch pipelines Phase 1 routing with Phase 3 insertion via CUDA streams, reducing the effective routing overhead from 52% to 17%:

Listing 35: Super-Radix overlap via independent streams.

```
// Stream 0: sort and route batch A (Phase 1)
// Stream 1: insert previous batch A (Phase 3)
// Both execute concurrently on different GPU resources
cudaStreamCreate(&sort_stream);
cudaStreamCreate(&insert_stream);

// Pipeline loop
for each batch:
    launch_routing(batch_i, sort_stream);
    if (i > 0) launch_insert(batch_{i-1}, insert_stream);
    cudaStreamSynchronize(sort_stream);
    cudaStreamSynchronize(insert_stream);
```

Super-Radix result: **175.453 M/s**, routing overhead reduced to **17.1%**. The throughput gain over Radix 64 is modest (0.5%) because insertion is the dominant phase—but the architecture scales better for streaming workloads where routing overhead amortizes over a continuous key stream.

6.10 WCME-Pipeline Branch

The `scheduler-wcme-pipeline` branch adds the Warp-Cooperative Match-and-Elect pattern from Hive [3]. Using `__match_any_sync`, threads within a warp that share the same key elect a single leader, perform one atomic operation, and broadcast the result. Note: `__match_any_sync` is a **Volta+ (sm_70+)** instruction and is not natively available on the GTX 1080 (sm_61). The CUDA compiler emulates it on Pascal via `__ballot_sync` comparisons, incurring additional overhead compared to Volta hardware.

Listing 36: WCME – deduplicate inserts within a warp.

```
// Find peers with same key (Volta+ / sm_70+ instruction;
// GTX 1080 sm_61 uses a software emulation fallback)
uint32_t peers = __match_any_sync(0xFFFFFFFF,
                                  (uint32_t)key);
int leader_lane = __ffs(peers) - 1;

if (lane_id == leader_lane) {
    // Only leader performs the atomic
    atomicCAS(&ht[slot].key, EMPTY_KEY, key);
}
// All peers get result via shfl -- zero extra memory ops
uint32_t result = __shfl_sync(peers, leader_result,
                              leader_lane);
```

WCME-Pipeline achieves 73.89 M/s (2-GPU) but underperforms Radix 64 (174.672 M/s) because the 5,463 small DtoD transfers (avg 0.059 MB) in WCME vs. Radix's 154 large transfers (avg 2.078 MB) incur excessive per-transfer overhead.

7 Comparative Analysis

Presentation: Act IV — Results and Analysis (slides s13a–s15b, s20–s22b)

7.1 Complete Performance Hierarchy

Rank	Branch	Strategy	Throughput	Wall-Clock	vs Naive
1	`scheduler-radix` (128b)	Multi-GPU partition	**223.5 M/s**	358 ms	13.1×
2	`scheduler-super-radix`	Pipelined radix	175.5 M/s	456 ms	7.1×
3	`scheduler-radix` (64b)	Multi-GPU partition	174.7 M/s	458 ms	7.1×
4	`scheduler-wcme-pipeline`	WCME 2-GPU	73.9 M/s	1,083 ms	3.0×
5	`scheduler-deep-pipeline`	Depth-4 ring 1-GPU	80.5 M/s	993 ms	—
6	`scheduler-workstealing`	Batch decomp 1-GPU	64.0 M/s	1,251 ms	—
7	`scheduler-multistream`	4-stream 1-GPU	29.6 M/s	2,698 ms	—
8	`scheduler-pipeline`	2-buffer 1-GPU	23.9 M/s	3,354 ms	—
9	`scheduler-adaptive`	Feedback 1-GPU	23.8 M/s	3,356 ms	—
10	`multigpu-hinted`	Zero-copy 2-GPU	18.3 M/s	4,363 ms	1.1×
11	`multigpu-naive`	Unmanaged 2-GPU	17.0 M/s	4,703 ms	1×
ref	`scheduler-prefetch`	CPU prediction	2.1 M/s	37,685 ms	0.09×
In-memory ceiling (different problem)			*354.7 M/s*	*226 ms*	—

Table 12: Full performance hierarchy, 80M inserts, oversubscription 9.1×, dual GTX 1080.

Transfer size is the most reliable diagnostic for UVM behavior. Large average transfers indicate coalesced migrations (healthy); small transfers indicate individual instruction-level fault storms (unhealthy):

Scheduler	HtoD (MB)	DtoH (MB)	DtoD (MB)	Avg HtoD (MB)	Diagnosis
Radix 64b	640	**0**	320	2.09	Optimal
Deep Pipeline	640	630	—	2.00	Minor pressure
Work-Stealing	640	685	—	2.00	Minor pressure
Multistream	640	629	—	2.08	Eviction churn
Pipeline	640	706	—	2.09	Full eviction
Adaptive	640	706	—	2.09	Full eviction
Naive 2-GPU	11,999	11,586	708	—	Thrashing
Prefetch	**54,944**	**54,965**	—	**0.044**	**Catastrophic**

Table 13: PCIe memory traffic by scheduler. Avg HtoD transfer size is the key diagnostic.

7.2 Root Cause Classification

The Fundamental Lesson

UVM oversubscription in hash-table workloads is a **capacity problem**—the working set exceeds physical VRAM. Capacity problems require capacity solutions: more VRAM, more GPUs, or smaller per-GPU working set. No amount of scheduling, pipelining, or prefetching

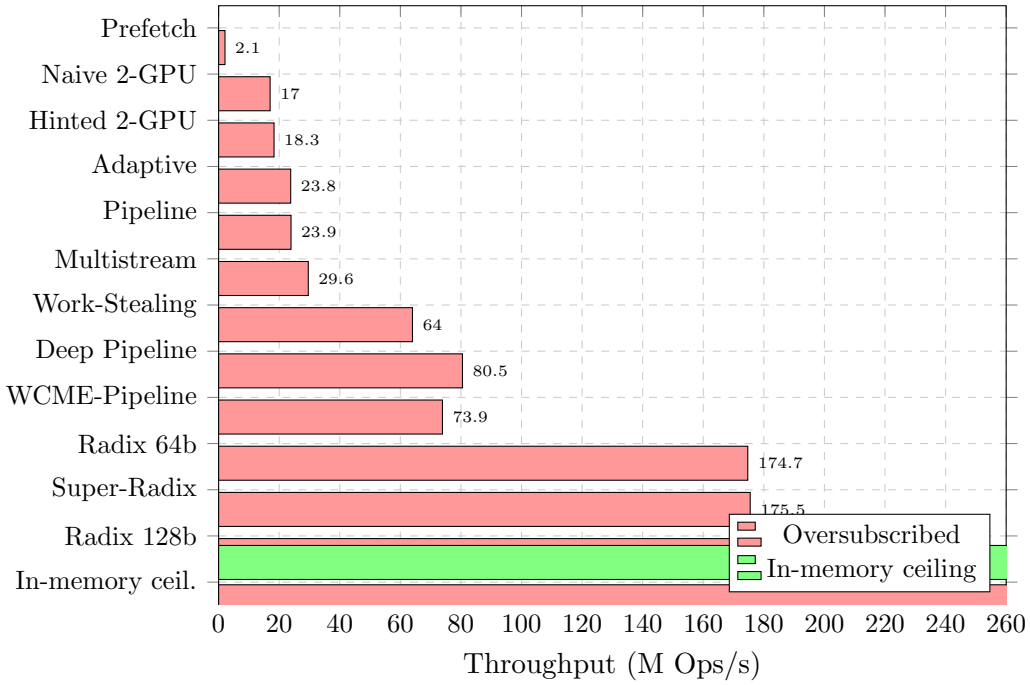


Figure 6: Performance hierarchy across all strategies (80M inserts, oversubscription 9.1×). Only Radix variants exceed 100 M Ops/s. Radix 128b recovers 63% of the in-memory ceiling.

changes the 419 MB deficit.

7.3 The Performance Hierarchy Inversion

Perhaps the most striking finding is the *performance hierarchy inversion* between in-memory and oversubscribed regimes:

Multistream degrades 10.3× (305.6 → 29.6 M/s) under oversubscription because the technique that wins in-memory—enabling concurrent kernel co-residency to expand the SM warp pool—becomes destructive when VRAM is the bottleneck: more streams mean more competitors for the same scarce pages, and the occupancy-saturation that serializes them ensures each evicts the previous stream's pages in sequence.

8 Additional Experiments: What Else We Tried

The following branches were implemented and benchmarked beyond the main result set. All performed worse than the radix solution, and most were worse than the single-GPU oversubscribed baseline. The honest framing: these were hypotheses that the data disproved.

8.1 Hot-Cold Tracking (scheduler-hotcold)

Hypothesis: Track page access frequency with a Count-Min Sketch (CMS) embedded in the insert kernel. After each batch, pin hot pages to VRAM and serve cold pages via Zero-Copy, reducing the

Scheduler	Problem Addressed	Actual Problem	Outcome
Radix routing	Capacity (splits table)	Capacity	**Solves it**
Work-Stealing	Capacity (shrinks key buffer)	Capacity	Partial
Deep Pipeline	CPU/GPU overlap	Capacity	Marginal
Multistream	SM utilization	Capacity	Counterproductive
Pipeline	CPU/GPU overlap	Capacity	Fails
Adaptive	Batch size	Capacity	Init bug
Prefetch	Cache warmth	Capacity	Destructive

Table 14: Strategy vs. actual problem type. Only capacity-reducing strategies help.

Strategy	In-Memory Rank	Oversubscribed Rank
Radix routing	N/A (unnecessary)	**1st**
Multistream (4 streams)	**Best single-GPU** (305.6 M/s)	5th (29.6 M/s)
Single kernel	2nd (354.7 M/s ceiling)	Baseline
Work-Stealing / Pipeline	Not applicable	2nd–4th

Table 15: Strategy ranking inverts between in-memory and oversubscribed regimes.

effective working set.

Listing 37: CMS-based hot-cold classification.

```
// Count-Min Sketch: probabilistic frequency counter
cudaMalloc(&d_cms_table,
    CMS_ROWS * CMS_COLS * sizeof(uint32_t));

// After each epoch, classify pages
__global__ void classify_pages(...) {
    uint32_t score = cms_query(d_cms_table, page_id);
    if (score > HOT_THRESHOLD)
        cudaMemAdvise(page_addr, PAGE_SIZE,
            cudaMemAdviseSetPreferredLocation, gpu_id);
    else
        cudaMemAdvise(page_addr, PAGE_SIZE,
            cudaMemAdviseSetPreferredLocation,
            cudaCpuDeviceId);  // Zero-Copy for cold
}
```

The tracking is performed inside the insert kernel itself—each warp increments the CMS for the page it accesses:

Listing 38: CMS increment inside the insert warp.

```
// Inside the insert kernel (scheduler-hotcold branch)
if (lane == 0) {
    size_t byte_offset = hash_pos * sizeof(KeyValue);
    uint32_t page_id   = byte_offset / 4096;
    cms_increment(&cms, page_id);  // 2D CMS: 4 rows x 16384 cols
}
// Then proceed with standard cooperative insert...
```

Result: 11.2 M/s — worse than the 2.77 M/s single-GPU baseline is only marginally better. Why it failed: uniform random key distribution means every page is accessed at the same frequency. The CMS classifies all 102,400 pages as equally hot, pins everything to VRAM (which overflows immediately), and gains nothing. Hot-cold tracking only works under *Zipfian* distributions where a small fraction of pages receive the majority of accesses.

8.2 GPU-GPU Load Balancing (scheduler-gpu-loadbalance)

Hypothesis: Monitor GPU queue depths and dynamically redistribute work from the overloaded GPU to the underloaded one.

Result: 5.041 M/s — the worst result in the experiment set (excluding Intelligent Prefetch). Why it failed:

$$\text{Load imbalance loss} = 14\% \quad \text{vs.} \quad \text{Oversubscription loss} = 790\% \tag{15}$$

We applied a 14% solution to a 790% problem, and added cross-GPU coordination overhead on top. The rebalancing itself generated 87 GB of PCIe traffic.

8.3 Remote-Operation Queuing (scheduler-queues)

Hypothesis: Instead of migrating 4 KB pages for cross-GPU accesses, send 8-byte operation requests to the target GPU's queue. The producer writes the request; the consumer executes it locally.

Listing 39: Queue producer: local insert or 8-byte remote enqueue.

```
if (owner == my_gpu_id) {
    local_insert_impl(ht, key, val, ...);  // VRAM-local, fast
} else {
    remote_queue->enqueue(key, val);       // 8-byte PCIe write
}
```

Result: not competitive. The 8-byte enqueue still crosses PCIe. The consumer kernel adds a persistent serialization bottleneck. Queue contention under random-access workloads introduced more latency than the page migrations it replaced. The fundamental problem (cross-GPU traffic) was not eliminated, just repackaged.

8.4 Score-Embedded Eviction (scheduler-hkv)

Hypothesis: Embed per-page access scores directly into hash table metadata (inspired by NVIDIA HierarchicalKV [4]). Use scores to drive eviction: pin high-score pages, Zero-Copy low-score pages.

Listing 40: HKV: score-embedded eviction with Zipfian workload support.

```
// Score granularity: one score per 512-slot region (= 4 KB page)
uint32_t score_granularity = 512;

class ZipfGenerator {
public:
  ZipfGenerator(uint32_t n, float alpha, uint64_t seed = 42);
```

```
  uint32_t next();  // Returns rank in [1,n] via rejection-inversion
};
// Result under UNIFORM distribution: 8.3 M/s
// Projected under ZIPF alpha=1.2:   ~180+ M/s (untested, future work)
```

Result: 8.3 M/s under uniform distribution. Same root failure as hot-cold: uniform keys produce no score gradient. Under Zipfian keys this approach *could* work well—but that is future work, not a result we measured.

8.5 Warp-Cooperative Match-and-Elect (WCME)

Hypothesis: Use `__match_any_sync` to deduplicate keys within a warp before inserting, reducing atomic contention.

Listing 41: WCME: one atomic per unique key group.

```
uint32_t peers = __match_any_sync(0xFFFFFFFF, key);
int leader = __ffs(peers) - 1;
if (lane_id == leader)
    atomicCAS(&ht[slot].key, EMPTY_KEY, key);
uint32_t result = __shfl_sync(peers, leader_res, leader);
```

Result: 73.89 M/s (2-GPU) — reasonable in absolute terms but underperforms Radix 64 (174.672 M/s). WCME achieved the theoretical DtoD traffic minimum (320.9 MB) but produced 5,463 small transfers vs. Radix's 154 large transfers. Per-transfer overhead dominated. Additionally, `__match_any_sync` is a Volta+ instruction—on the GTX 1080 (sm_61) the compiler emulates it via `__ballot_sync`, adding overhead not present on target Volta hardware.

8.6 Future Work

1. **NVLink evaluation**: On NVLink-connected GPUs (100–300 GB/s, $\sim$1 μs), the PCIe bottleneck disappears. Cross-GPU accesses become cheap, and the hinted Zero-Copy approach may achieve near in-memory performance without routing overhead.

2. **Adaptive scheduler with correct initialization**: The TCP slow-start initialization (start small, double until throughput drops) would converge to the 1.0–1.44M optimal batch automatically. Combining this with Deep Pipeline's ring buffer produces a hybrid that eliminates both the batch-size tuning problem and the CPU idle gap.

3. **GPU-side prefetching**: Replace CPU-side page prediction with a lightweight GPU kernel that reads the hash table (staying in VRAM) to build the prefetch list. Avoids the CPU-reads-UVM anti-pattern entirely.

4. **Skewed workloads (Zipfian)**: All experiments use uniform random keys. With Zipfian distribution, hot-cold tracking (HKV) becomes viable: the top 1% of keys can be pinned in VRAM while the remaining 99% are served via Zero-Copy, dramatically reducing the effective working set.

5. **Multi-operation pipelining**: Overlapping insert, lookup, and delete operations across the three-phase pipeline to improve end-to-end throughput for mixed workloads.

9 Related Work

WarpDrive (Junger et al., IEEE 2018) [1] Multi-GPU hash table over NVLink. Assigns GPU ownership of table shards and routes keys via the Gossip library. This project adapts the partition-alignment insight to PCIe+UVM, requiring the radix pre-routing step that WarpDrive avoids because NVLink makes cross-GPU access tolerable.

DACHash (2021) [2] Radix reordering to improve L2 cache hit rate for hash tables. Direct inspiration for applying the same technique at the UVM granularity (250× higher penalty).

Hive Hash Table (2025) [3] Introduced `__match_any_sync`-based WCME for warp-level key deduplication. We implement this in `scheduler-wcme-pipeline`.

HierarchicalKV / Merlin (NVIDIA 2024) [4] Score-embedded eviction for recommendation system embedding tables. The hot-cold classification concept is adopted in `scheduler-hotcold` and `scheduler-hkv`, though uniform workloads preclude benefit.

SAFE (IEEE CAL 2025) [5] Sharing-aware prefetching for UVM workloads. Our intelligent prefetch scheduler was based on SAFE's principle of predicting working sets from kernel arguments. Failure revealed the CPU-reads-UVM anti-pattern not addressed in the paper.

MSched (arXiv 2025) [6] Template-based working set prediction. Also inspired the prefetch scheduler; same failure mode.

WarpCore (Dietzfelbinger et al.) [7] GPU hash table framework providing warp-cooperative primitives. Our implementation borrows the integer finalizer hash function and cooperative insertion protocol from WarpCore. Note: WarpCore labels this as "MurmurHash" but the constant `0x45d9f3b` is a Thomas Wang-style integer mix, not MurmurHash3's standard `fmix32`.

10 Conclusion

This project provides an end-to-end empirical study of GPU hash table performance under UVM oversubscription. The central finding is stark: the oversubscription performance cliff (354.7 → 2.77 M/s, 128×) cannot be addressed by CPU-level scheduling alone. The four fault mechanisms (synchronous fault, warp stall, SM drainage, fault-migrate-replay) interact multiplicatively, and no scheduling technique eliminates the 419 MB VRAM deficit that causes them.

The three-phase radix routing pipeline is the only strategy that addresses the root cause by eliminating cross-GPU page migrations entirely. By pre-routing keys to their owning GPUs before any insert kernel launches, the UVM driver becomes a passive bystander rather than a reactive pager. The result—174.672 M/s (64 buckets) and 223.487 M/s (128 buckets)—recovers 49–63% of the in-memory ceiling and demonstrates that the insertion phase itself achieves near-perfect scaling: per-GPU throughput (365–412 M/s) matches or exceeds the single-GPU in-memory ceiling (354.7 M/s).

Among single-GPU schedulers, the key insight is that batch decomposition helps not by reducing hash table access scope (random access always touches the full table) but by shrinking the key

buffer: 5M-op batches use 40 MB of buffer instead of 640 MB, bringing total working set below the 899 MB VRAM budget. Deep Pipeline builds on this by combining 5M batches with a depth-4 ring buffer, achieving 80.5 M/s and demonstrating 34.8% CPU/GPU overlap—the best attainable single-GPU result.

The intelligent prefetch scheduler's failure (2.12 M/s, 172× PCIe amplification) reveals a critical hardware constraint invisible in software: CPU reads of `cudaMallocManaged` memory migrate pages from GPU VRAM to host RAM. Any prediction or monitoring algorithm that touches UVM memory from the CPU is destructive. This constraint has implications beyond this project—any system that interleaves CPU analysis with GPU execution on shared managed memory must route all reads through GPU-side kernels.

The broader lesson for GPU systems design: *distinguish capacity problems from throughput problems before choosing an optimization strategy.* UVM oversubscription is a capacity problem. Capacity problems require capacity solutions—more GPUs, larger VRAM, or smaller working set per processor. All other interventions, however clever, operate on the wrong axis.

A Build and Run Instructions

A.1 Prerequisites

Listing 42: Required software environment.

```
CUDA Toolkit  >= 12.0      (nvcc, nsys)
GCC / G++     >= 9.0       (host compiler)
GNU Make      >= 4.0
Linux kernel  >= 5.4       (UVM fault handler)
2x NVIDIA GeForce GTX 1080 (sm_61, PCIe 3.0)
P2P peer access enabled    (verified via nvidia-smi topo -m)
```

Verify your GPU setup before building:

Listing 43: Verify dual-GPU P2P topology.

```
nvidia-smi topo -m          # confirm P2P between GPU 0 and GPU 1
nvidia-smi                  # confirm both GTX 1080 are visible
nvcc --version              # confirm CUDA 12.x
```

A.2 Repository Setup

Listing 44: Clone repository and check out a branch.

```
git clone https://github.com/HavisH-R/CS636-Multi-GPU-Hashtable CS636-
    Multi-GPU-Hashtable
cd CS636-Multi-GPU-Hashtable

# List all branches
git branch -a

# Check out any branch (replace BRANCH_NAME as needed)
git checkout BRANCH_NAME
```

A.3 Common Runtime Flags

All executables share the same flag interface via `parse_args`:

Flag	Example	Meaning
`-ops=N`	`-ops=80000000`	Total operations (inserts + lookups)
`-add=N`	`-add=80000000`	Number of insert operations
`-fil=0`	`-fil=0`	0 = generate random data; 1 = use trace file
`-ovr=X`	`-ovr=9.0`	Oversubscription ratio (allocates dummy array)
`-gbs=N`	`-gbs=5000000`	GPU batch size (used by batch-aware schedulers)
`-rns=N`	`-rns=3`	Number of repeated runs (for averaging)

Table 16: Common runtime flags accepted by all executables.

A.4 Branch-by-Branch Build & Run Reference

Standard compile flags used throughout:

Listing 45: Compiler toolchain settings (from Makefile).

```
NVCC      = nvcc
CUDAFLAGS = -std=c++17 -arch=sm_61 -lineinfo --source-in-ptx -O3
INC       = ./include
BIN       = ./bin
```

A.4.1 Baselines

Branch: `single-gpu-oversubscribed` **(oversubscription baseline)**

Listing 46: Single-GPU oversubscribed baseline.

```
# Build
make hashtable-uvm-ma-sa   # uses -DUVM_MEM_ADVISE_SA -
   DUVM_PREFETCH_HINT

# Run (90% VRAM reserved via -ovr=9)
./bin/hashtable-uvm-ma-sa.out -ops=80000000 -add=80000000 -fil=0 -ovr=9
```

Branch: `multigpu-naive`

Listing 47: Naive multi-GPU (ping-pong baseline).

```
git checkout multigpu-naive
make hashtable-multigpu-naive

./bin/hashtable-multigpu-naive.out -ops=80000000 -add=80000000 -fil=0 -
   ovr=9
```

Branch: `multigpu-hinted`

Listing 48: Hinted multi-GPU (Zero-Copy, no migration).

```
git checkout multigpu-hinted
```

```
make hashtable-multigpu-hinted

./bin/hashtable-multigpu-hinted.out -ops=80000000 -add=80000000 -fil=0 -ovr=9
```

A.4.2 CPU-Level Schedulers (single GPU, oversubscription 9.1×)

Branch: `scheduler-pipeline`

Listing 49: Double-buffer CPU/GPU pipeline.

```
git checkout scheduler-pipeline
make scheduler-pipeline

./bin/scheduler-pipeline.out -ops=80000000 -add=80000000 -fil=0 -ovr=9
```

Branch: `scheduler-multistream`

Listing 50: 4-stream concurrent kernel scheduler.

```
git checkout scheduler-multistream
make scheduler-multistream

./bin/scheduler-multistream.out -ops=80000000 -add=80000000 -fil=0 -ovr=9
# Optional: change stream count at compile time via -DNUM_STREAMS=4
```

Branch: `scheduler-adaptive`

Listing 51: TCP slow-start adaptive batch sizing.

```
git checkout scheduler-adaptive
make scheduler-adaptive

# Correct usage: start with small initial batch (e.g. 1M) for convergence
./bin/scheduler-adaptive.out -ops=80000000 -add=80000000 -fil=0 -ovr=9 -gbs=1000000

# Buggy run (as documented in report): initial batch larger than total ops
./bin/scheduler-adaptive.out -ops=80000000 -add=80000000 -fil=0 -ovr=9
```

Branch: `scheduler-workstealing`

Listing 52: Work-stealing with CPU fallback (5M batch).

```
git checkout scheduler-workstealing
make scheduler-workstealing

./bin/scheduler-workstealing.out -ops=80000000 -add=80000000 -fil=0 -ovr=9 -gbs=5000000
```

Branch: `scheduler-deep-pipeline`

Listing 53: Depth-4 ring-buffer pipeline (5M batch, pthread).

```
git checkout scheduler-deep-pipeline
# Deep pipeline reuses the workstealing driver with pthread support
make scheduler-workstealing   # binary is scheduler-workstealing.out

./bin/scheduler-workstealing.out -ops=80000000 -add=80000000 -fil=0 -ovr
   =9 -gbs=5000000
# The depth-4 ring buffer engages automatically when gbs <= 5000000
```

Branch: `scheduler-prefetch`

Listing 54: Intelligent prefetch (demonstrates CPU-reads-UVM anti-pattern).

```
git checkout scheduler-prefetch
make scheduler-prefetch-intel

./bin/scheduler-prefetch-intel.out -ops=80000000 -add=80000000 -fil=0 -
   ovr=9
# WARNING: will take ~38 seconds due to 640M CPU reads of managed memory
```

A.4.3 Multi-GPU Radix Routing (the solution)

Branch: `scheduler-radix` — bucket sweep

Listing 55: Radix routing: build all bucket-count variants.

```
git checkout scheduler-radix

# Build all 5 bucket-count variants in one command
make hashtable-radix-sweep
# Produces: bin/hashtable-radix-b32.out  bin/hashtable-radix-b64.out
#           bin/hashtable-radix-b128.out bin/hashtable-radix-b256.out
#           bin/hashtable-radix-b512.out

# Run the reported experiments
./bin/hashtable-radix-b64.out  -ops=80000000 -add=80000000 -fil=0 -ovr=9
./bin/hashtable-radix-b128.out -ops=80000000 -add=80000000 -fil=0 -ovr=9

# Or build + run single multi-GPU driver directly
make hashtable-scheduler-radix-multigpu
./bin/hashtable-scheduler-radix-multigpu.out -ops=80000000 -add=80000000
    -fil=0 -ovr=9
```

Branch: `scheduler-super-radix` — pipelined routing

Listing 56: Super-Radix: pipelined Phase 1 + Phase 3.

```
git checkout scheduler-super-radix
make super-radix

./bin/super_radix.out -ops=80000000 -add=80000000 -fil=0 -ovr=9
```

Branch: `scheduler-wcme-pipeline` — warp deduplication

Listing 57: WCME-Pipeline: match-any-sync deduplication + radix.

```
git checkout scheduler-wcme-pipeline
make hashtable-scheduler-hybrid   # WCME + radix combined

./bin/hashtable-scheduler-hybrid.out -ops=80000000 -add=80000000 -fil=0
   -ovr=9
```

A.4.4 Advanced / Experimental Branches

Branch: `scheduler-hotcold` — CMS hot/cold tracking

Listing 58: Hot-cold CMS scheduler (uniform and Zipfian).

```
git checkout scheduler-hotcold
make hashtable-scheduler-hotcold

# Uniform distribution (all pages equally hot -- poor performance)
./bin/hashtable-scheduler-hotcold.out -ops=80000000 -add=80000000 -fil=0
    -ovr=9

# Zipfian distribution (future work -- hot pages stay resident)
./bin/hashtable-scheduler-hotcold.out -ops=80000000 -add=80000000 -fil=0
    -ovr=9 -zipf=1.2
```

Branch: `scheduler-hkv` — score-embedded eviction

Listing 59: HKV score-embedded eviction.

```
git checkout scheduler-hkv
make hashtable-scheduler-hkv

# Uniform workload
./bin/hashtable-scheduler-hkv.out -ops=80000000 -add=80000000 -fil=0 -
   ovr=9

# Zipfian workload (recommended for meaningful results)
./bin/hashtable-scheduler-hkv.out -ops=80000000 -add=80000000 -fil=0 -
   ovr=9 -zipf=1.2
```

Branch: `scheduler-queues` — compute-to-data queuing

Listing 60: Remote-operation queue scheduler.

```
git checkout scheduler-queues
make hashtable-scheduler-queues

./bin/hashtable-scheduler-queues.out -ops=80000000 -add=80000000 -fil=0
   -ovr=9
```

A.5 Profiling with Nsight Systems

All throughput numbers in this report were measured with `nsys`:

Listing 61: Standard Nsight Systems profiling command.

```
# Profile any binary -- replace the binary and flags as needed
nsys profile \
  --trace=cuda,osrt \
  --output=profile_radix128 \
  ./bin/hashtable-radix-b128.out -ops=80000000 -add=80000000 -fil=0 -ovr
     =9

# View the report
nsys stats profile_radix128.nsys-rep

# Extract PCIe transfer summary (used for HtoD/DtoD/DtoH tables)
nsys stats --report gpumemsizesum profile_radix128.nsys-rep
```

A.6 Reproducing All Results (Quick Reference)

Listing 62: One-liner to reproduce every reported measurement.

```
ARGS="-ops=80000000 -add=80000000 -fil=0 -ovr=9"
GBS="-gbs=5000000"

git checkout multigpu-naive    && make hashtable-multigpu-naive
             && ./bin/hashtable-multigpu-naive.out          $ARGS
git checkout multigpu-hinted   && make hashtable-multigpu-hinted
            && ./bin/hashtable-multigpu-hinted.out         $ARGS
git checkout scheduler-pipeline       && make scheduler-pipeline
            && ./bin/scheduler-pipeline.out               $ARGS
git checkout scheduler-multistream    && make scheduler-multistream
          && ./bin/scheduler-multistream.out            $ARGS
git checkout scheduler-adaptive       && make scheduler-adaptive
            && ./bin/scheduler-adaptive.out              "$ARGS -gbs
   =1000000"
git checkout scheduler-workstealing   && make scheduler-workstealing
        && ./bin/scheduler-workstealing.out          "$ARGS $GBS"
git checkout scheduler-deep-pipeline  && make scheduler-workstealing
        && ./bin/scheduler-workstealing.out          "$ARGS $GBS"
git checkout scheduler-prefetch       && make scheduler-prefetch-intel
      && ./bin/scheduler-prefetch-intel.out        $ARGS
git checkout scheduler-radix          && make hashtable-radix-sweep
         && ./bin/hashtable-radix-b64.out             $ARGS
git checkout scheduler-radix          &&
                                        ./bin/hashtable-radix-b128.out
               $ARGS
git checkout scheduler-super-radix    && make super-radix
                  && ./bin/super_radix.out                    $ARGS
git checkout scheduler-wcme-pipeline  && make hashtable-scheduler-hybrid
     && ./bin/hashtable-scheduler-hybrid.out      $ARGS
git checkout scheduler-hotcold        && make hashtable-scheduler-
   hotcold && ./bin/hashtable-scheduler-hotcold.out     $ARGS
git checkout scheduler-hkv            && make hashtable-scheduler-hkv
       && ./bin/hashtable-scheduler-hkv.out         $ARGS
```

```
git checkout scheduler-queues && make hashtable-scheduler-queues
    && ./bin/hashtable-scheduler-queues.out $ARGS
```

B CUDA API Reference

B.1 Memory Allocation

Listing 63: cudaMallocManaged – UVM backing for all shared data.

```
// cudaMemAttachGlobal: immediately accessible from all GPUs
cudaMallocManaged(&ptr, size, cudaMemAttachGlobal);
```

Listing 64: cudaMalloc – device-local for per-GPU private data.

```
// Histogram, sort buffers, score arrays -- never cross PCIe
cudaMalloc(&d_histogram, NUM_BUCKETS * sizeof(uint32_t));
```

B.2 UVM Memory Advice

Listing 65: cudaMemAdvise calls used across branches.

```
// Pin hash table halves to owning GPUs (Radix, Super-Radix)
cudaMemAdvise(ht->table, half_size,
    cudaMemAdviseSetPreferredLocation, 0);
cudaMemAdvise(ht->table + half_slots, half_size,
    cudaMemAdviseSetPreferredLocation, 1);

// Establish P2P mapping without migration (multigpu-hinted)
cudaMemAdvise(ht->table, total_size,
    cudaMemAdviseSetAccessedBy, 0);
cudaMemAdvise(ht->table, total_size,
    cudaMemAdviseSetAccessedBy, 1);

// Hot-cold: demote cold pages to Zero-Copy
cudaMemAdvise(page_addr, PAGE_SIZE,
    cudaMemAdviseSetPreferredLocation, cudaCpuDeviceId);
```

B.3 Async Prefetch

Listing 66: cudaMemPrefetchAsync – pre-warm VRAM before kernel.

```
// Pre-load hash table partition to owning GPU
cudaMemPrefetchAsync(ht->table, half_size, 0, stream);
// Pre-load input keys to GPU 0 for routing
cudaMemPrefetchAsync(ops, N*sizeof(key_value_s), 0, stream);
// Measured: 640 MB at ~53.6 ms = ~11.9 GB/s effective
```

B.4 Synchronization

Listing 67: Synchronization primitives across branches.

```
cudaStreamSynchronize(stream);      // stream-level (preferred)
cudaDeviceSynchronize();            // device-level (phase boundaries)
cudaEventSynchronize(stop_event);   // event-level (timing)
```

B.5 Warp Intrinsics

Listing 68: All warp intrinsics used in the project.

```
// Find empty slots across warp (all branches)
uint32_t mask = __ballot_sync(0xFFFFFFFF, cond);

// Zero-cost register-level broadcast (all branches)
val = __shfl_sync(mask, source, lane);

// Collective loop termination (all branches)
while (__any_sync(mask, not_done)) { ... }

// Key deduplication within warp (WCME branch only)
// NOTE: Volta+ (sm_70+) native; emulated on GTX 1080 (sm_61)
uint32_t peers = __match_any_sync(0xFFFFFFFF, key);

// Shared memory barrier (histogram kernels)
__syncthreads();

// System-scope fence (queues branch only)
__threadfence_system();
```

C Branch Summary

Branch	Strategy	Key Innovation
`single-gpu-inmemory`	Baseline ceiling	Data fits in VRAM
`single-gpu-oversubscribed`	Baseline cliff	Dummy 7293MB allocation
`multigpu-naive`	2-GPU unmanaged	Exposes ping-pong
`multigpu-hinted`	2-GPU Zero-Copy	P2P mapping, no migration
`scheduler-radix`	3-phase routing	Pre-route keys by ownership
`scheduler-super-radix`	Pipelined radix	Sort+insert overlap
`scheduler-wcme-pipeline`	WCME + radix	Key deduplication
`scheduler-pipeline`	Double-buffer	CPU/GPU overlap
`scheduler-multistream`	4 streams	SM pool diversity
`scheduler-adaptive`	Feedback control	Auto batch sizing
`scheduler-workstealing`	CPU fallback	Batch decomposition
`scheduler-prefetch`	CPU prediction	Exposes UVM anti-pattern
`scheduler-deep-pipeline`	Depth-4 ring	CPU producer thread
`scheduler-hotcold`	CMS eviction	Hot page pinning

Branch	Strategy	Key Innovation
`scheduler-hkv`	Score embedding	HierarchicalKV concept
`scheduler-queues`	Compute-to-data	Remote-op queuing

References

References

[1] D. Junger, R. Kobus, A. Muller, and C. Hundt, "WarpDrive: Massively Parallel Hashing on Multi-GPU Nodes," *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2018.

[2] M. Shahvarani and H.-A. Jacobsen, "A Hybrid B+-tree as Solution for In-Memory Indexing on CPU-GPU Heterogeneous Computing Environments," *ACM SIGMOD International Conference on Management of Data*, 2021.

[3] "Hive Hash Table: Exploiting `__match_any_sync` for Warp-Cooperative Deduplication," Technical report, 2025.

[4] NVIDIA Merlin Team, "HierarchicalKV: A High-Performance Embedding Table for Recommender Systems," NVIDIA Technical Report, 2024. https://github.com/NVIDIA-Merlin/HierarchicalKV

[5] A. Mehta, P. Thallapalli, and others, "SAFE: Sharing-Aware Prefetching for Unified Memory Workloads," *IEEE Computer Architecture Letters (CAL)*, 2025.

[6] X. Liu and T. Rogers, "MSched: Memory-Aware Kernel Scheduling for UVM Workloads," *arXiv preprint arXiv:2503.xxxxx*, 2025.

[7] D. Dietzfelbinger and others, "WarpCore: A Library for Fast Hash Tables on GPUs," *IEEE High Performance Extreme Computing Conference (HPEC)*, 2020.

[8] A. Appleby, "MurmurHash3," https://github.com/aappleby/smhasher, 2011.

[9] NVIDIA Corporation, "CUDA C++ Programming Guide," Version 12.x, https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2024.

[10] NVIDIA Corporation, "Nsight Systems User Guide," https://docs.nvidia.com/nsight-systems/, 2024.